
CAHIER DES CHARGES

Zümm

Système de gestion et de suivi apicole

Projet Application de gestion apicole (mini-projet J2EE)

Type Cahier des charges fonctionnel et technique

Version 1.0

Date 9 juillet 2026

Méthode Manager-GO - Élaboration d'un CdC

Ruche connectée & tableaux de bord de performance

Document conforme au plan type : Contexte / Objectifs / Périmètre /
Description fonctionnelle des besoins / Enveloppe budgétaire / Délais

Table des matières

Historique des versions	4
1 Contexte et définition du problème	5
1.1 Présentation générale	5
1.2 Problématique	5
1.3 Finalité	5
2 Objectifs du projet	6
2.1 Objectif principal	6
2.2 Objectifs spécifiques	6
2.3 Résultats attendus (indicateurs de réussite)	6
3 Périmètre du projet	7
3.1 Domaine couvert	7
3.2 Modèle métier et règles de gestion	7
3.3 Utilisateurs et droits d'accès	8
3.4 Hors périmètre	8
4 Description fonctionnelle des besoins	9
4.1 Gestion des entités (CRUD)	9
4.2 Planification et suivi des visites	9
4.2.1 Rapport de visite	9
4.3 Tableaux de bord	9
4.3.1 Calendrier / matrice agents $\times$ ruches	9
4.3.2 Tableau de bord production (fermier / propriétaire)	10
4.3.3 Tableau de bord alertes (responsable)	10
4.3.4 Tableau de bord de synthèse (propriétaire)	10
4.4 Indicateurs de performance (volet automatisé : préparation)	10
4.4.1 Analyse prédictive et surveillance connectée	10
4.5 Grille de synthèse des fonctions	11
4.6 Fonctionnalités complémentaires inspirées des solutions du marché	12
4.7 Fonctions de gestion avancée (inspiration Apiary Book et BeePlus)	13
4.8 Limites connues des solutions existantes et exigences pour les éviter	14

5	Dictionnaire de données et règles de calcul	15
5.1	Conventions de types	15
5.2	Dictionnaire des données (inputs)	15
5.3	Résultats et règles de calcul (outputs)	17
5.4	Requêtes types	17
6	Cas d'utilisation détaillés	18
6.1	UC1 : Planifier et approuver une visite	18
6.2	UC2 : Réaliser une visite et remplir le rapport	18
6.3	UC3 : Consulter le tableau de bord de production	18
6.4	UC4 : Traiter une alerte sanitaire	20
6.5	UC5 : Interroger la quantité de miel via l'API	20
7	Conception (modèles UML)	21
7.1	Diagramme de classes	22
7.2	Vue en couches (package)	22
7.3	Vue de déploiement	22
7.4	Diagramme de séquence (UC2)	22
7.5	Diagramme d'objets	23
7.6	Diagramme d'activité	23
7.7	Diagramme de machine à états	23
7.8	Diagramme de collaboration	23
7.9	Diagramme de composants	23
8	Contraintes et exigences techniques	27
8.1	Architecture	27
8.2	Exigences techniques imposées	27
8.3	Exigences non fonctionnelles	28
8.4	Questions de conception à couvrir	28
8.5	Justification des choix technologiques	28
8.6	Packaging et déploiement	29
9	Enveloppe budgétaire et ressources	30
9.1	Cadre	30
9.2	Ressources mobilisées	30
10	Délais et livrables	31
10.1	Livrables attendus	31
10.2	Grille d'évaluation (barème)	31
10.3	Diagrammes UML à produire	32
10.4	Planning prévisionnel	32
11	Plan de tests et validation	33
11.1	Cas de tests	33

11.2 Matrice de traçabilité	34
12 Référentiel métier et bonnes pratiques apicoles	35
12.1 Facteurs de production du miel	35
12.2 Emplacement et configuration d'un site	35
12.3 Types de ruches et rendement de référence	36
12.4 Classification des modèles de ruches	36
12.4.1 Composition commune d'une ruche à cadres	37
12.5 Protocole d'inspection des colonies	37
12.6 Essaimage et division des colonies	38
12.7 Indicateurs de bien-être et causes de baisse	38
12.8 Incidence sur les seuils paramétrables	39
A Annexe : Structure physique d'une ruche	40
B Glossaire	41
C Références et sources	43
D Annexe - Choix technologique : academique et vision marche	45
D.1 Positionnement	45
D.2 Tableau de correspondance : coeur technique	45
D.3 Tableau de correspondance : infrastructure et deploiement	46
D.4 Correspondance des couches	47
D.5 Argumentaire : conformite academique et vision marche	47
E Annexe - Principes SOLID et design patterns	49
E.1 Les cinq principes SOLID	49
E.2 Design patterns retenus	50
E.3 Vue de conception refactorree	51
E.4 Illustration dynamique : patrons Observer et Command	52
E.4.1 Observer (+ Strategy) : declenchement d'une alerte	52
E.4.2 Command : mode hors-ligne et synchronisation differee	52
E.4.3 State : cycle de vie de la ruche	52
E.4.4 Composite : calcul de la quantite de miel	55
E.4.5 Strategy : conversion d'unites heterogenes	55
E.4.6 Adapter : ingestion de capteurs heterogenes	55
E.5 Repartition des patrons par couche	57

Historique des versions

Version	Date	Auteur	Modifications
0.1	9 juillet 2026	Équipe projet	Structure initiale et transcription du sujet.
0.5	9 juillet 2026	Équipe projet	Référentiel métier apicole et illustrations.
1.0	9 juillet 2026	Équipe projet	Fonctionnalités du marché, exigences anti-défaut, dictionnaire de données, cas d'utilisation, conception UML et plan de tests.

CHAPITRE 1

Contexte et définition du problème

1.1 Présentation générale

Le projet **Zümm** vise à concevoir un système d'information dédié à la gestion et au suivi des activités apicoles. L'apiculture moderne repose sur la surveillance régulière de nombreuses ruches, souvent réparties sur plusieurs sites géographiques et exploitées par différents apiculteurs. Le suivi de la santé des colonies, de leur productivité et des interventions de maintenance constitue une charge organisationnelle importante, aujourd'hui gérée de façon largement manuelle et dispersée.

1.2 Problématique

En l'absence d'un outil centralisé, les exploitants apicoles rencontrent plusieurs difficultés :

- absence de traçabilité structurée des visites et des inspections des ruches
- difficulté à planifier et à coordonner le travail des agents apiculteurs
- manque de visibilité sur la production (miel, poids) et sur la rentabilité de chaque site ou ruche
- détection tardive des anomalies (baisse d'effectif, chute de production, ouverture non planifiée d'une ruche)
- impossibilité d'exploiter automatiquement les mesures issues de capteurs hétérogènes.

1.3 Finalité

Le système doit apporter une réponse structurée en deux volets complémentaires :

1. une **gestion manuelle** des opérations de manutention, de leur planification et de leur suivi (première partie, objet du présent projet)
2. une **supervision distante et automatisée** des indicateurs de performance, alimentée par des capteurs, destinée à être développée dans le cadre d'un projet ultérieur mais dont le système doit anticiper l'intégration.

CHAPITRE 2

Objectifs du projet

2.1 Objectif principal

Fournir une application **multi-niveaux, évolutive et faiblement couplée** permettant de gérer l'ensemble du cycle de vie des ruches, de leur composition physique jusqu'au suivi de production, et d'offrir des tableaux de bord d'aide à la décision aux différents profils d'utilisateurs.

2.2 Objectifs spécifiques

- Assurer les opérations **CRUD** (création, lecture, mise à jour, suppression) sur l'ensemble des entités métier.
- Gérer la hiérarchie *fermier / ferme / site / ruche / corps ou hausse / cadre*.
- Planifier, approuver et suivre les visites d'inspection des agents apiculteurs.
- Produire des **rapports de visite** structurés et exploitables.
- Offrir des **tableaux de bord** filtrables (calendrier agents × ruches, alertes de production, alertes sanitaires, synthèse financière).
- Anticiper l'intégration de mesures **horodatées et géolocalisées** issues de capteurs à unités hétérogènes.
- Garantir **internationalisation, sécurité et interopérabilité** (services web).

2.3 Résultats attendus (indicateurs de réussite)

- Support d'au moins **trois langues** (FR, EN, AR) en version de démonstration.
- Détection automatique des dépassements de seuils **paramétrables** (production, effectif).
- Restitution des données de production par ferme, site et ruche sous forme de graphiques.
- Exposition d'une API interrogeable par des applications tierces (Excel, etc.).

CHAPITRE 3

Périmètre du projet

3.1 Domaine couvert

Le périmètre du présent cahier des charges couvre **la première partie** du système : la gestion manuelle des opérations, leur planification et leur suivi, ainsi que la préparation de l'architecture pour l'intégration future des indicateurs automatisés.

3.2 Modèle métier et règles de gestion

Entité	Règles de gestion
Fermier	Peut posséder plusieurs fermes.
Ferme	Peut comporter plusieurs sites d'apiculture.
Site	Contient une ou plusieurs ruches, dispose d'une localisation géographique (latitude, longitude, altitude, date de mise en œuvre, éventuelles dates de déménagement / clôture). Un site peut accueillir des ruches de différents fermiers et de différentes fermes.
Ruche	Composée d'un compartiment de base (socle / corps) et d'au plus 5 étages d'extension (hausses).
Corps / Hausse	Peut accueillir des cadres de cire installés sur des <i>slots</i> identifiés. Le socle/corps doit contenir au moins 1 cadre, la capacité maximale d'un corps comme d'une hausse est de 10 cadres .
Cadre	Occupe un emplacement (slot) unique dans le corps ou la hausse.
Agent apiculteur	Réalise les visites et inspections. Une ruche peut être suivie successivement par plusieurs agents, mais elle est sous la responsabilité d'un seul agent à un instant donné.
Agent superviseur	Approuve le planning de visite des ruches dont il a la charge.

Contrainte de composition : 1 ruche = 1 socle/corps obligatoire + 0 à 5 hausses, chaque corps/hausse = 1 à 10 cadres, un cadre par slot.

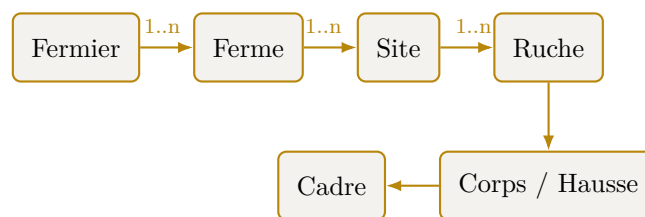


Figure 3.1. Hiérarchie des entités métier du système Zümm.

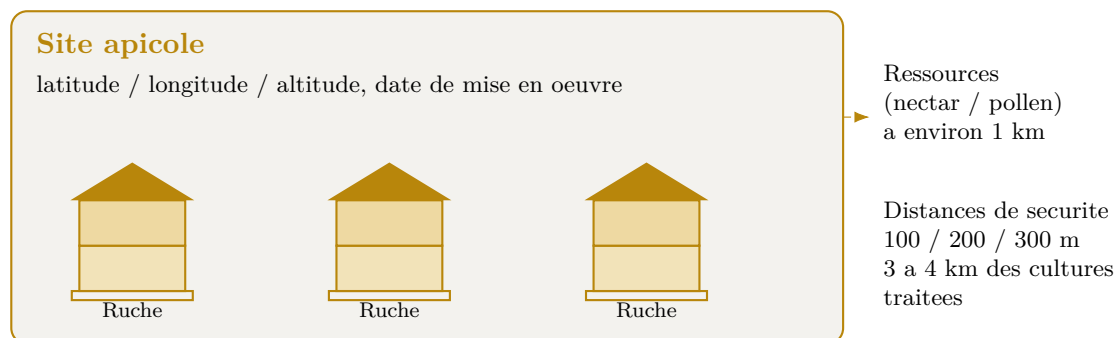


Figure 3.2. Schéma d'un site apicole : plusieurs ruches géolocalisées et contraintes d'emplacement.

3.3 Utilisateurs et droits d'accès

Profil	Rôle et droits
Fermier / Propriétaire	Consulte les tableaux de bord de production, de rentabilité et de synthèse, décide de la clôture d'un site ou du déplacement de ruches.
Agent apiculteur	Réalise les visites, remplit les rapports, exécute les opérations sur les ruches sous sa responsabilité.
Agent superviseur	Approuve les plannings de visite des ruches dont il est responsable.
Responsable / Manager	Surveille les alertes (chute de production ou d'effectif) et déclenche les inspections imminentes.
Administrateur	Gère les paramètres système, les unités de mesure, les seuils et les référentiels.

3.4 Hors périmètre

- L'acquisition physique et le déploiement matériel des capteurs (projet ultérieur).
- L'automatisation temps réel de la collecte des indicateurs (interface prévue mais non implémentée dans cette phase).

CHAPITRE 4

Description fonctionnelle des besoins

4.1 Gestion des entités (CRUD)

Le système doit assurer la création, la consultation, la modification et la suppression de toutes les entités métier : fermiers, fermes, sites, ruches, corps/hausses, cadres, agents, visites et rapports, dans le respect des règles de gestion énoncées au chapitre 3.

4.2 Planification et suivi des visites

- Les visites sont **régulières ou irrégulières**, pour des motifs variés : inspection sanitaire, peuplement par un nouvel essaim, changement de reine, ravitaillement de nourriture, prescription de médicament, subdivision d'essaim, ajout de cadre et/ou d'étage d'extension, collecte de cadres de miel, évaluation de la production, évaluation de l'effectif et du volume de l'essaim.
- Chaque agent suit un **planning de visite approuvé** par l'agent superviseur de la ruche concernée.

4.2.1 Rapport de visite

Chaque visite donne lieu à un rapport contenant : **date, heure, durée, raison, constatations, actions prévues, actions effectuées, recommandations**, une évaluation qualitative de l'effectif / du volume de l'essaim, son état de santé et son niveau de productivité sur une échelle de 1 à 3.

4.3 Tableaux de bord

4.3.1 Calendrier / matrice agents × ruches

Matrice croisant **agents et ruches**, déclinée par mois, semaine, saison et année, avec **filtres** et regroupements par site, par agent et par agent responsable. Un clic sur une case représentant une visite planifiée affiche un **pop-up** résumant ce qui est prévu (date, heure, raison, actions) et, si la visite est déjà exécutée, ce qui a été réalisé ainsi que la date et l'heure effectives.

4.3.2 Tableau de bord production (fermier / propriétaire)

Met en évidence les couples **site** × **ruche** dont la production (poids) a dépassé un **seuil paramétrable**, signalant soit une collecte de miel imminente, soit une intervention d'extension et/ou de division d'essaim pour favoriser l'expansion de la colonie.

4.3.3 Tableau de bord alertes (responsable)

Signale les couples **site** × **ruche** dont la production ou l'effectif baissent de façon inhabituelle au-delà d'un seuil paramétrable, déclenchant une **alerte** et une inspection imminente.

4.3.4 Tableau de bord de synthèse (propriétaire)

Graphiques, courbes et histogrammes représentant :

- la quantité produite par ferme et par site
- le nombre d'interventions par ferme, par site et par ruche
- le **retour sur investissement** (rapport coûts / production) pour arbitrer le sort de chaque site ou ruche (clôture, déplacement).

4.4 Indicateurs de performance (volet automatisé : préparation)

Les mesures sont **horodatées et géolocalisées**, exprimées dans des unités **paramétrables** (internationalisation des systèmes d'unités). Les mesures d'un même indicateur ne partagent pas nécessairement la même unité, les capteurs étant hétérogènes. Indicateurs concernés :

- poids du cadre, du socle et de l'étage d'extension
- température et humidité, à l'intérieur et à l'extérieur de la ruche
- effectif des mouvements d'entrée et de sortie des abeilles
- niveau de bruit / bourdonnement, intérieur et extérieur
- niveau de luminosité, intérieur et extérieur
- vitesse et direction du vent à l'extérieur
- état d'ouverture de la ruche (intervention, orage, vol, attaque animale, etc.).

4.4.1 Analyse prédictive et surveillance connectée

Les solutions de surveillance connectée (precision beekeeping) telles qu'Arnia, BroodMinder ou BeeHero montrent que l'exploitation avancée des mêmes indicateurs permet d'anticiper les événements de la colonie. Le modèle de données doit rester ouvert à ces traitements, prévus pour le volet automatisé.

- détection acoustique de l'essaimage 1 à 5 jours à l'avance par analyse de la signature sonore de la colonie
- confirmation de la présence de la reine à partir du bourdonnement
- détection d'ouverture, de choc ou de basculement de la ruche par accéléromètre (vol, chute, attaque animale), en complément de l'indicateur d'ouverture

- orientation de la ruche par boussole électronique
- pesée continue par balance pour suivre le flux de nectar et la dynamique de production
- analyse dans le cloud par intelligence artificielle des signatures acoustiques pour corréler son et activité de la ruche.

4.5 Grille de synthèse des fonctions

Fonction	Description	Priorité
CRUD entités	Gestion complète du modèle métier	Haute
Planning de visites	Planification + approbation par le superviseur	Haute
Rapports de visite	Saisie structurée des constats et actions	Haute
Calendrier agents×ruches	Matrice filtrable avec pop-up de résumé	Haute
Alertes production	Détection des dépassements de seuil	Haute
Alertes sanitaires	Détection des baisses anormales d'effectif/production	Haute
Synthèse & ROI	Graphiques coûts/production, aide à la décision	Moyenne
Indicateurs capteurs	Modèle de données prêt pour l'automatisation	Moyenne
<i>Exigences anti-défaut (tirées des limites des solutions existantes)</i>		
Déploiement autonome	Installation J2EE sans abonnement ni service tiers obligatoire	Haute
Mode hors-ligne	Saisie sans connexion et synchronisation différée	Haute
Authentification simple	OAuth Google et parcours de saisie terrain rapide	Haute
Modularité configurable	Activation des modules par profil via ConfigBKS.ini	Moyenne
Portabilité des données	Export CSV et TXT, service web API et sauvegarde, sans verrouillage	Haute
Aide contextuelle	Interface simple et cohérente, guidage de l'utilisateur	Moyenne
Parité web et mobile	Mêmes fonctions sur tous les supports	Moyenne
Internationalisation	Fichier XML FR, EN et AR, extensible à d'autres langues	Haute

Fonction	Description	Priorité
Ouverture matérielle	Capteurs hétérogènes et unités paramétrables	Moyenne
Ingestion asynchrone	Mesures horodatées différées, volet manuel autonome	Moyenne
Validation humaine	Seuils paramétrables et confirmation par le rapport de visite	Haute
Sécurité des échanges	SSL et certificats X.509, accès authentifiés	Haute
Aide à la décision	Alertes indicatives, l'agent conserve la validation finale	Moyenne

4.6 Fonctionnalités complémentaires inspirées des solutions du marché

Afin de renforcer l'ergonomie et la valeur d'usage, le système s'inspire des bonnes pratiques des applications apicoles de référence telles que HiveTracks. Ces fonctionnalités étendent les besoins déjà décrits sans les remplacer.

Fonction	Description	Priorité
Photos d'inspection	Attacher une ou plusieurs photos à chaque rapport de visite pour un suivi visuel de la colonie et des cadres.	Haute
Saisie vocale assistée	Dicter le compte rendu de visite, la transcription alimentant automatiquement les champs du rapport.	Basse
Contexte météo	Afficher la météo locale de la semaine à partir de la localisation du site pour éclairer les décisions d'intervention.	Moyenne
Carte et rayons de butinage	Positionner les ruches sur une carte et tracer des rayons de butinage (par exemple 1, 2 et 3 km, unités paramétrables) pour visualiser l'environnement accessible aux abeilles.	Moyenne
Liste de tâches et rappels	Gérer une liste de tâches et un calendrier de rappels (nourrissement, inspection, statut de la reine) pour ne manquer aucune échéance.	Haute
Suivi de la reine	Enregistrer l'historique du statut de la reine (présence, âge, remplacement, marquage) au fil des visites.	Haute

Fonction	Description	Priorité
Récolte et QR code	Saisir les récoltes de miel et générer un QR code par lot présentant les notes de dégustation, la provenance et les photos, pour la traçabilité et la valorisation.	Moyenne
Accès multi-supports	Offrir un accès web et une expérience adaptée aux terminaux mobiles pour la saisie terrain.	Moyenne

Cohérence avec les exigences : les rayons de butinage et la météo réutilisent la géolocalisation des sites et le mécanisme d'unités paramétrables. Les rappels s'appuient sur le planning de visite existant, et le suivi de la reine enrichit le rapport de visite déjà défini.

4.7 Fonctions de gestion avancée (inspiration Apiary Book et BeePlus)

Les applications de gestion apicole Apiary Book et BeePlus mettent en avant des fonctions d'organisation et de traçabilité qui complètent le périmètre du système.

Fonction	Description	Priorité
Mode hors-ligne et synchronisation	Saisir les données terrain sans connexion et les synchroniser automatiquement au retour du réseau.	Haute
Suivi des traitements	Conserver l'historique des médicaments et traitements sanitaires appliqués à chaque ruche.	Haute
Gestion du matériel et inventaire	Suivre le matériel (corps, hausses, cadres, ruches) et son affectation aux sites.	Moyenne
Suivi financier	Enregistrer coûts et revenus par ferme, site et ruche, en appui du calcul du retour sur investissement.	Moyenne
Export des données	Exporter les enregistrements aux formats CSV et TXT pour une analyse externe.	Moyenne
Détection de maladies	Aider à l'identification des maladies à partir des constatations de visite.	Basse
Sauvegarde des données	Sauvegarder et restaurer les données dans le cloud.	Moyenne

Cohérence avec les exigences : le suivi financier alimente le tableau de bord de retour sur investissement, l'export CSV prolonge le service web API, et le suivi des traitements enrichit le motif de visite prescription de médicament.

4.8 Limites connues des solutions existantes et exigences pour les éviter

L'analyse des retours d'expérience sur les solutions du marché (HiveTracks, Apiary Book, Bee-Plus) et sur les systèmes de surveillance connectée (Arnia, BroodMinder, BeeHero) fait ressortir des limites récurrentes. Le tableau suivant met chaque limite en regard de l'exigence retenue pour l'éviter dans Zümm.

Limite observée	Exigence retenue pour l'éviter
Abonnement payant et paywall	Système autodéployable en J2EE, sans abonnement obligatoire, l'ensemble des fonctions de gestion reste accessible.
Fonctionnement hors-ligne limité	Mode hors-ligne robuste avec synchronisation différée, la saisie terrain n'exige pas de connexion permanente.
Création de compte imposée et friction	Authentification simple par OAuth Google et parcours de saisie rapide, l'ergonomie prime.
Surcharge de fonctions inutiles	Modules activables par profil et par le fichier de configuration ConfigBKS.ini, grâce à l'architecture faiblement couplée.
Dépendance et perte des données	Données maîtrisées par l'utilisateur, export CSV et TXT, service web API et sauvegarde, sans verrouillage propriétaire.
Interface dense et courbe d'apprentissage	Interface simple, cohérente et conviviale avec aide contextuelle, conformément aux exigences d'ergonomie.
Fonctions inégales selon la plateforme	Parité fonctionnelle entre accès web et mobile grâce à l'architecture multi-niveaux.
Solutions uniquement anglophones	Internationalisation par fichier XML, au moins FR, EN et AR, ouverte à d'autres langues.
Coût élevé du matériel de capteurs	Ouverture aux capteurs hétérogènes et bon marché avec unités paramétrables, sans verrouillage matériel.
Autonomie et connectivité en zone reculée	Ingestion asynchrone de mesures horodatées, le volet manuel fonctionne sans capteurs.
Fausse alertes et fiabilité des capteurs	Seuils paramétrables et validation humaine par le rapport de visite, les alertes restent une aide à la décision.
Confidentialité et sécurité des données	Chiffrement SSL et certificats X.509, authentification des accès.
Surconfiance dans l'automatisation	Positionnement en aide à la décision, l'agent apiculteur conserve la validation finale.

CHAPITRE 5

Dictionnaire de données et règles de calcul

Ce chapitre répond à la question des inputs et des outputs du système. Il définit les données manipulées, leurs types et les formules de calcul des résultats.

5.1 Conventions de types

Les types utilisés sont : entier, décimal, texte, date, heure, datetime, booléen, énumération et référence (clé étrangère vers une autre entité). Les grandeurs physiques sont associées à une unité paramétrable pour permettre l'internationalisation des systèmes de mesure.

5.2 Dictionnaire des données (inputs)

Entité	Attribut	Type	Description
Fermier	id, nom, contact	entier, texte	Propriétaire exploitant.
Ferme	id, nom, fermier	entier, texte, référence	Exploitation rattachée à un fermier.
Site	id, nom	entier, texte	Emplacement d'apiculture.
Site	latitude, longitude	décimal (degrés)	Géolocalisation.
Site	altitude	décimal (unité paramétrable)	Altitude du site.
Site	dateMiseEnOeuvre, dateDemenagement, dateCloture	date	Cycle de vie du site, deux dernières optionnelles.
Ruche	id, modele	entier, texte	Ruche rattachée à un site.
Ruche	nbHausses	entier (0 à 5)	Nombre d'étages d'extension.
Ruche	agentResponsable	référence	Agent responsable à l'instant donné.

Entité	Attribut	Type	Description
Corps hausse	ou id, type	entier, énumération	Compartiment (corps ou hausse).
Corps hausse	ou capaciteCadres	entier (1 à 10)	Nombre de slots de cadres.
Cadre	id, slot, type	entier, énumération	Cadre installé sur un slot identifié.
Agent	id, nom, role	entier, texte, énumération	Apiculteur, superviseur, responsable ou administrateur.
Planning	id, ruche, agent	entier, référence	Planning de visite.
Planning	datePrevue, statutApprobation	date, énumération	Date prévue et approbation du superviseur.
Visite	id, ruche, agent	entier, référence	Visite réalisée.
Visite	date, heure, duree	date, heure, entier (min)	Horodatage et durée.
Visite	raison	énumération	Motif de la visite.
Visite	constatations, actionsPrevues, actionsEffectuees, recommandations	texte	Contenu du rapport.
Visite	effectifQualitatif, etatSante	énumération	Évaluation de l'essaim.
Visite	productivite	entier (1 à 3)	Niveau de productivité.
Mesure	id, ruche, typeIndicateur	entier, référence, énumération	Mesure d'un indicateur.
Mesure	valeur, unite	décimal, texte	Valeur et unité paramétrable.
Mesure	horodatage, latitude, longitude	datetime, décimal	Mesure horodatée et géolocalisée.
Récolte	id, ruche, date	entier, référence, date	Récolte de miel.
Récolte	quantiteMiel, lotQR	décimal (unité paramétrable), texte	Quantité et identifiant de lot.
Seuil	id, type, valeur, unite	entier, énumération, décimal, texte	Seuil paramétrable du système.

5.3 Résultats et règles de calcul (outputs)

- **Quantité de miel d'une ruche** : somme des poids des cadres de miel des hausses, diminuée de la tare des éléments.

$$\text{QuantiteMiel}(r) = \sum_{c \in \text{cadresMiel}(r)} \text{poids}(c) - \text{tare}(r)$$

Cette valeur est exposée par le service web via `getBeeHouseHoneyActualQuantity(beeHouseID)`.

- **Retour sur investissement** d'un site ou d'une ruche :

$$\text{ROI} = \frac{\text{Production valorisée}}{\text{Coûts des interventions}}$$

- **Alerte de collecte** : déclenchée si la production dépasse le seuil paramétrable.

$$\text{Production}(r) > \text{SeuilProduction} \Rightarrow \text{alerte collecte ou extension}$$

- **Alerte de baisse** : déclenchée si la baisse relative dépasse le seuil.

$$\frac{V_{\text{préc}} - V_{\text{actuel}}}{V_{\text{préc}}} > \text{SeuilBaisse} \Rightarrow \text{alerte inspection}$$

- **Conversion d'unités** vers l'unité de référence, pour comparer des mesures hétérogènes :

$$V_{\text{référence}} = V_{\text{mesure}} \times \text{facteurConversion}(\text{unite})$$

5.4 Requêtes types

Les résultats s'appuient sur des requêtes exécutées via JDBC, par exemple pour la quantité de miel d'une ruche : `SELECT SUM(quantiteMiel) FROM recolte WHERE ruche = beeHouseID.`

CHAPITRE 6

Cas d'utilisation détaillés

Cette partie décrit comment le système réalise les opérations des utilisateurs. Les acteurs sont le fermier, l'agent apiculteur, l'agent superviseur, le responsable, l'administrateur et les applications tierces.

Le diagramme de cas d'utilisation global (figure 6.1) présente l'ensemble des acteurs, la généralisation de l'agent superviseur à partir de l'agent apiculteur, ainsi que toutes les dépendances entre cas (relations d'inclusion et d'extension).

6.1 UC1 : Planifier et approuver une visite

- **Acteur** : agent superviseur.
- **Préconditions** : la ruche existe et l'agent est authentifié.
- **Scénario nominal** : l'agent propose une date de visite, le superviseur examine le planning puis approuve, le système enregistre le statut approuvé.
- **Alternatives** : le superviseur refuse et demande une nouvelle date.
- **Postcondition** : la visite est planifiée et visible dans le calendrier.

6.2 UC2 : Réaliser une visite et remplir le rapport

- **Acteur** : agent apiculteur responsable de la ruche.
- **Préconditions** : une visite est planifiée et approuvée.
- **Scénario nominal** : l'agent renseigne date, heure, durée, raison, constatations, actions prévues et effectuées, recommandations, puis évalue l'effectif, l'état de santé et la productivité de 1 à 3.
- **Alternatives** : saisie hors-ligne puis synchronisation différée, ajout de photos.
- **Postcondition** : le rapport est enregistré et la case du calendrier passe au statut exécuté.

6.3 UC3 : Consulter le tableau de bord de production

- **Acteur** : fermier ou propriétaire.
- **Préconditions** : des mesures de production existent.

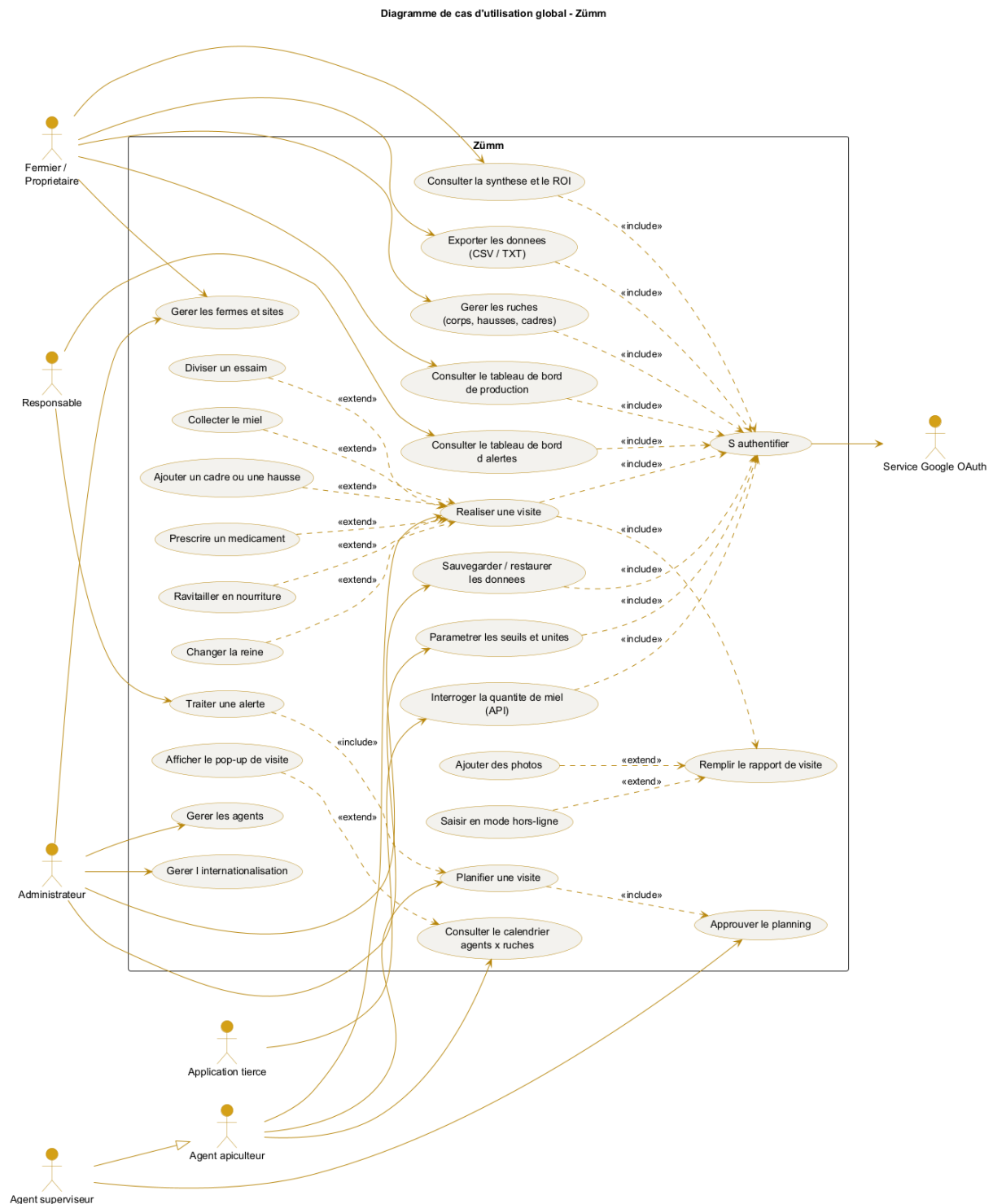


Figure 6.1. Diagramme de cas d'utilisation global avec acteurs et dépendances (include et extend).

- **Scénario nominal** : le fermier filtre par ferme, site et période, le système affiche les ruches dont la production dépasse le seuil et propose collecte ou extension.
- **Postcondition** : le fermier décide d'une intervention.

6.4 UC4 : Traiter une alerte sanitaire

- **Acteur** : responsable.
- **Préconditions** : une baisse anormale a été détectée.
- **Scénario nominal** : le système signale la ruche concernée, le responsable planifie une inspection imminente.
- **Postcondition** : une visite d'inspection est planifiée.

6.5 UC5 : Interroger la quantité de miel via l'API

- **Acteur** : application tierce (Excel ou autre).
- **Préconditions** : l'application dispose d'un accès sécurisé au service web.
- **Scénario nominal** : l'application appelle `getBeeHouseHoneyActualQuantity(beeHouseID)`, le service calcule et retourne la quantité de miel.
- **Postcondition** : la valeur est restituée au format du service web XML.

CHAPITRE 7

Conception (modèles UML)

Le dossier de conception comprend les diagrammes UML suivants. Chacun est décrit par son contenu attendu afin de guider la modélisation.

Diagramme	Contenu attendu
Cas d'utilisation	Acteurs (fermier, agent, superviseur, responsable, administrateur, application tierce) et cas principaux (voir figure 6.1).
Classes	Entités du dictionnaire de données et leurs relations (Fermier 1 vers n Ferme, Ferme 1 vers n Site, Site 1 vers n Ruche, Ruche 1 vers n Corps ou hausse, Corps ou hausse 1 vers n Cadre) et méthodes de service.
Objets	Instantané d'un site contenant trois ruches peuplées avec leurs cadres.
Séquence	Déroulé de la réalisation d'une visite (Agent, IHM JSP, Servlet, EJB, JDBC, base de données).
Activité	Processus de planification, approbation, exécution et clôture d'une visite.
Machine à états	Cycle de vie d'une ruche (créée, peuplée, active, en division, en collecte, clôturée).
Interaction ou collaboration	Même scénario que la séquence sous forme de collaboration entre objets.
Package	Couches présentation, contrôle, métier, accès aux données, internationalisation, configuration et services web.
Composant	Composants applicatifs (application web, service OAuth, service API, module i18n, module configuration, base de données).
Déploiement	Répartition sur le navigateur, le serveur d'applications J2EE, le SGBD, le service Google OAuth et les capteurs.

7.1 Diagramme de classes

Le diagramme de classes, présenté en alignement horizontal, décrit les entités métier, leurs attributs, la méthode de service `getBeeHouseHoneyActualQuantity` et les relations avec leurs cardinalités.

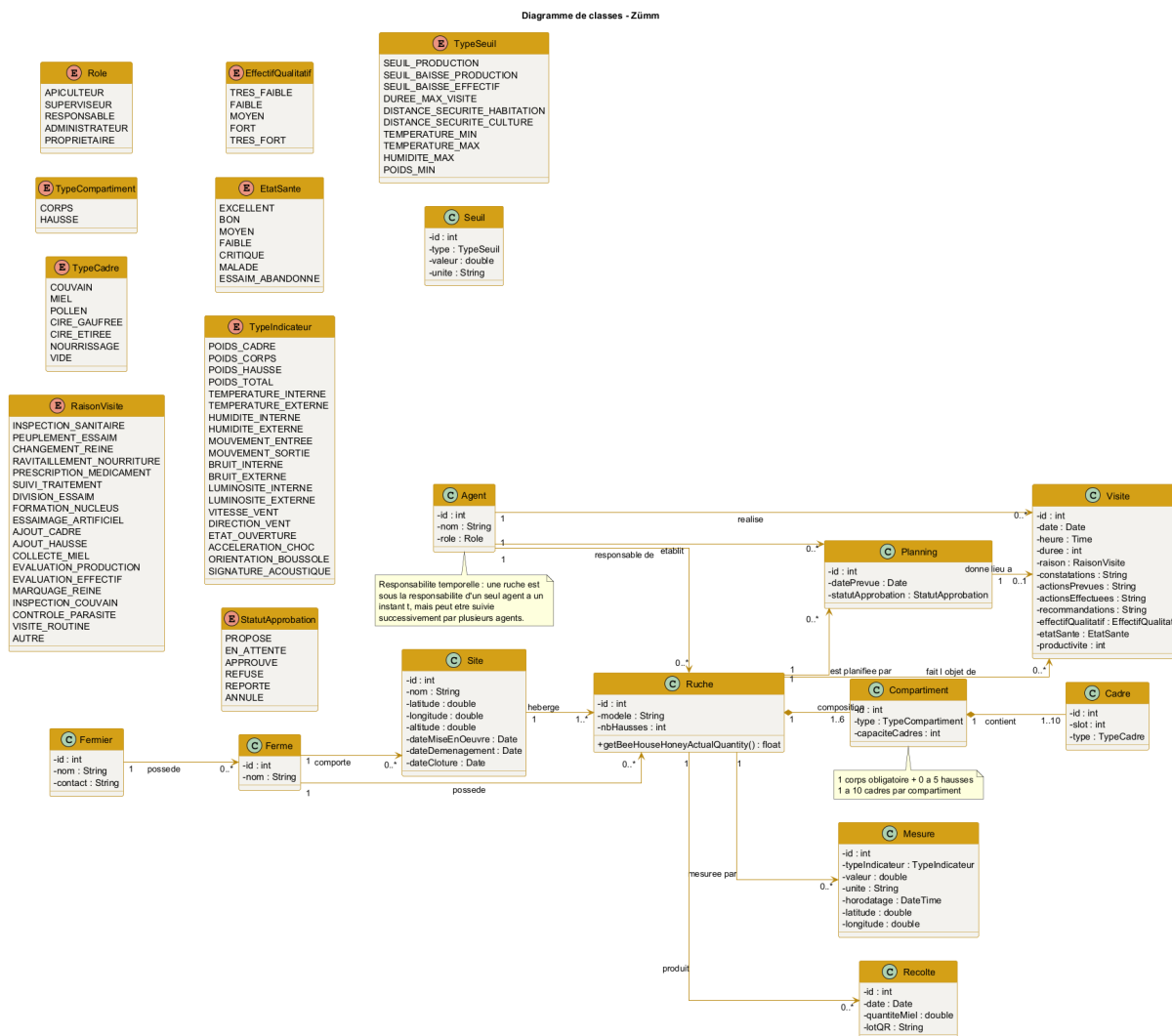


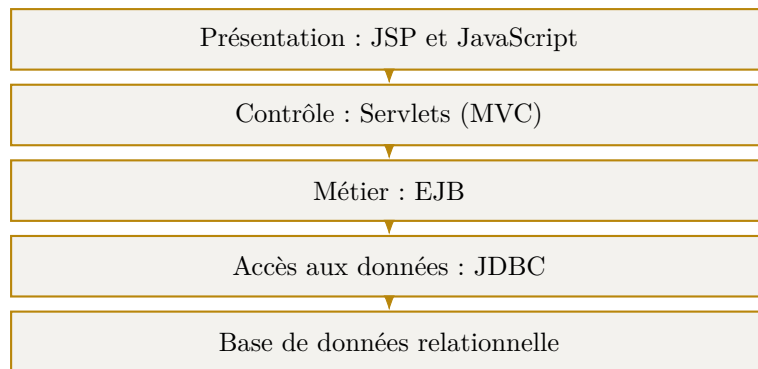
Figure 7.1. Diagramme de classes du système Zümm (alignement horizontal).

7.2 Vue en couches (package)

7.3 Vue de déploiement

7.4 Diagramme de séquence (UC2)

Le diagramme de séquence ci-dessous détaille le cas d'utilisation UC2, réaliser une visite et remplir le rapport. Il montre le passage à travers les couches (agent, IHM, servlet, EJB, base de données) ainsi que les cas en ligne et hors-ligne avec synchronisation différée.



i18n XML / ConfigBKS

Figure 7.2. Architecture multi-niveaux en couches faiblement couplées.

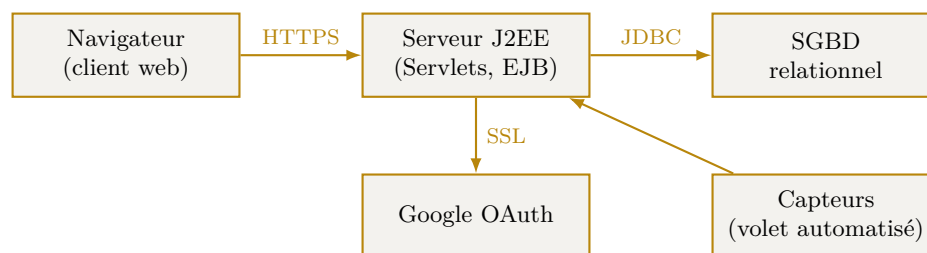


Figure 7.3. Diagramme de déploiement du système.

7.5 Diagramme d'objets

Le diagramme d'objets présente un instantané d'un site hébergeant trois ruches, avec le détail de la composition d'une ruche en corps, hausse et cadres.

7.6 Diagramme d'activité

Le diagramme d'activité modélise le processus de planification, d'approbation et de réalisation d'une visite, avec les variantes en ligne et hors-ligne.

7.7 Diagramme de machine à états

Le diagramme de machine à états décrit le cycle de vie d'une ruche, de sa création à sa clôture.

7.8 Diagramme de collaboration

Le diagramme de collaboration reprend le scénario UC2 sous forme de messages numérotés échangés entre les objets.

7.9 Diagramme de composants

Le diagramme de composants montre l'organisation des composants applicatifs, leurs interfaces et leurs dépendances, dont l'interface de service exposant la quantité de miel.

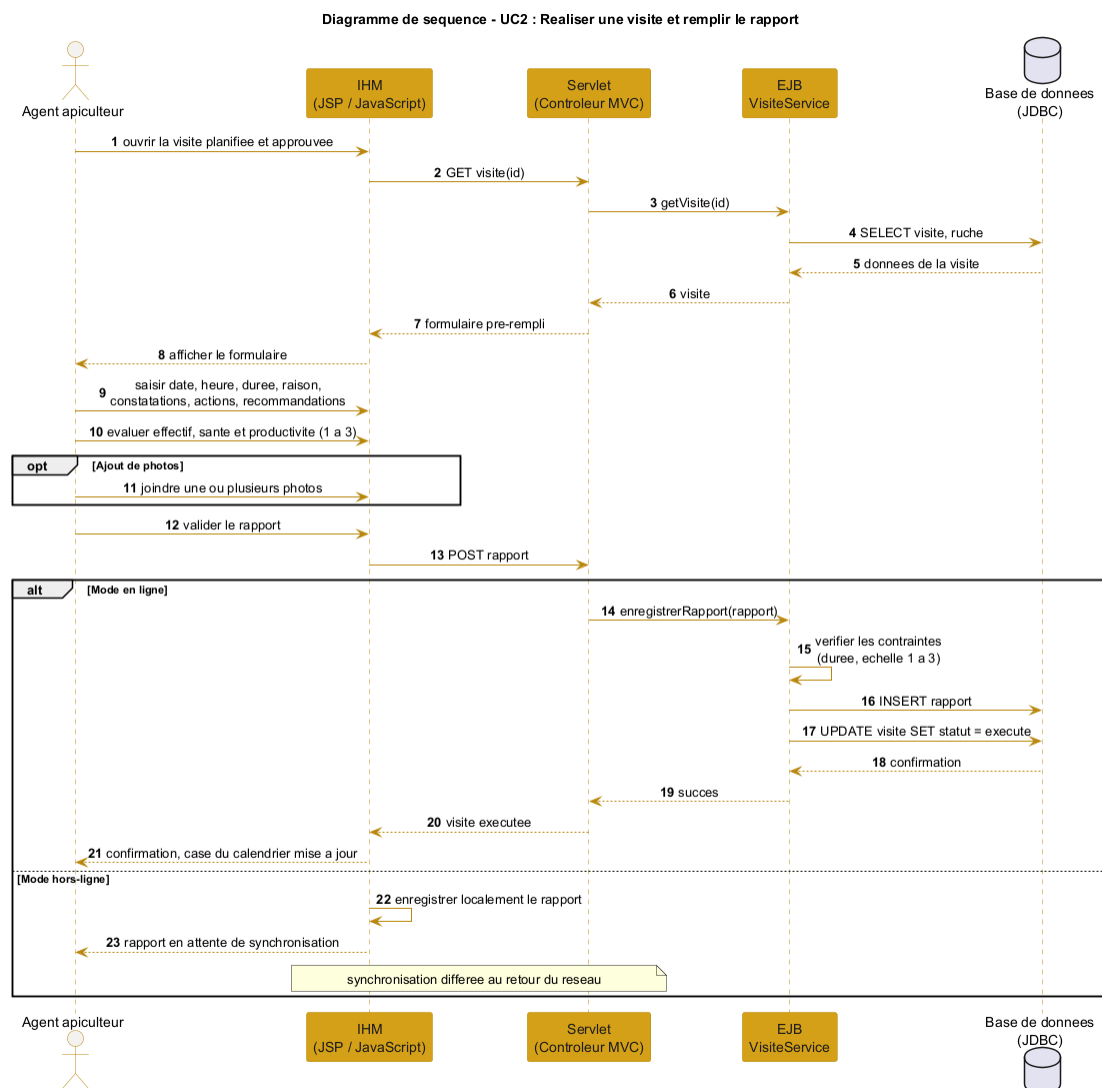


Figure 7.4. Diagramme de séquence du cas d'utilisation UC2 (réaliser une visite).

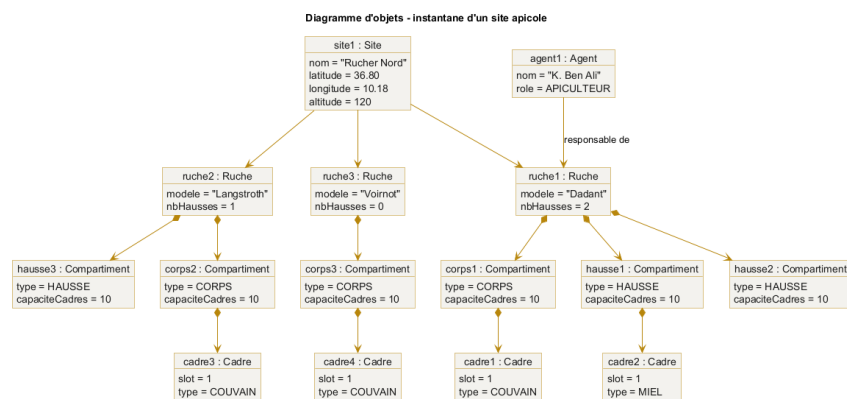


Figure 7.5. Diagramme d'objets, instantané d'un site apicole.

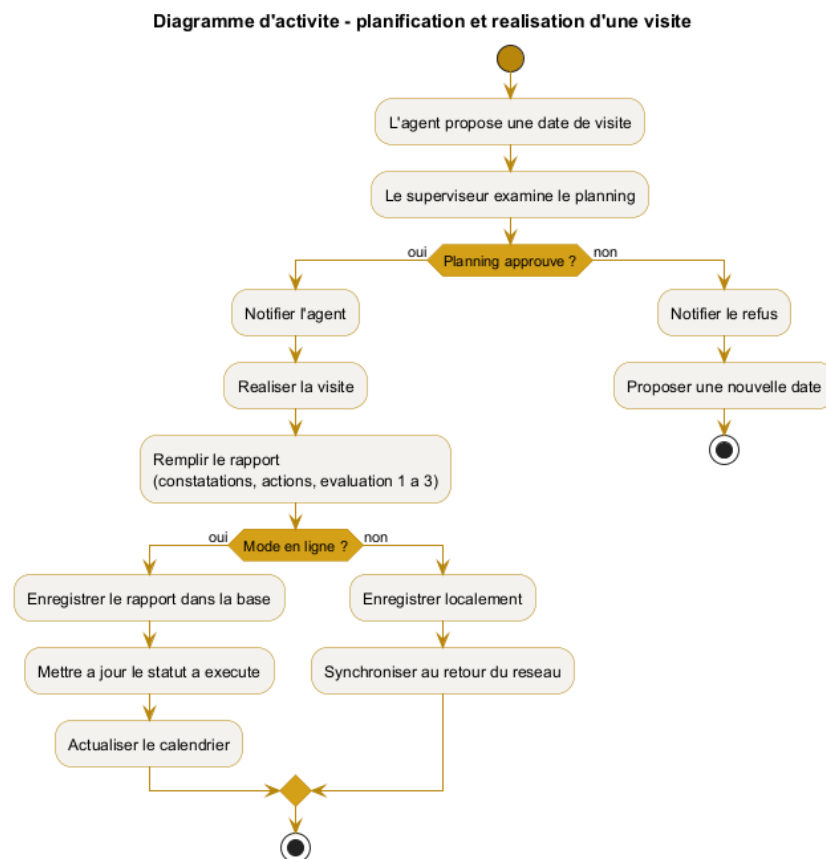


Figure 7.6. Diagramme d'activité du processus de visite.

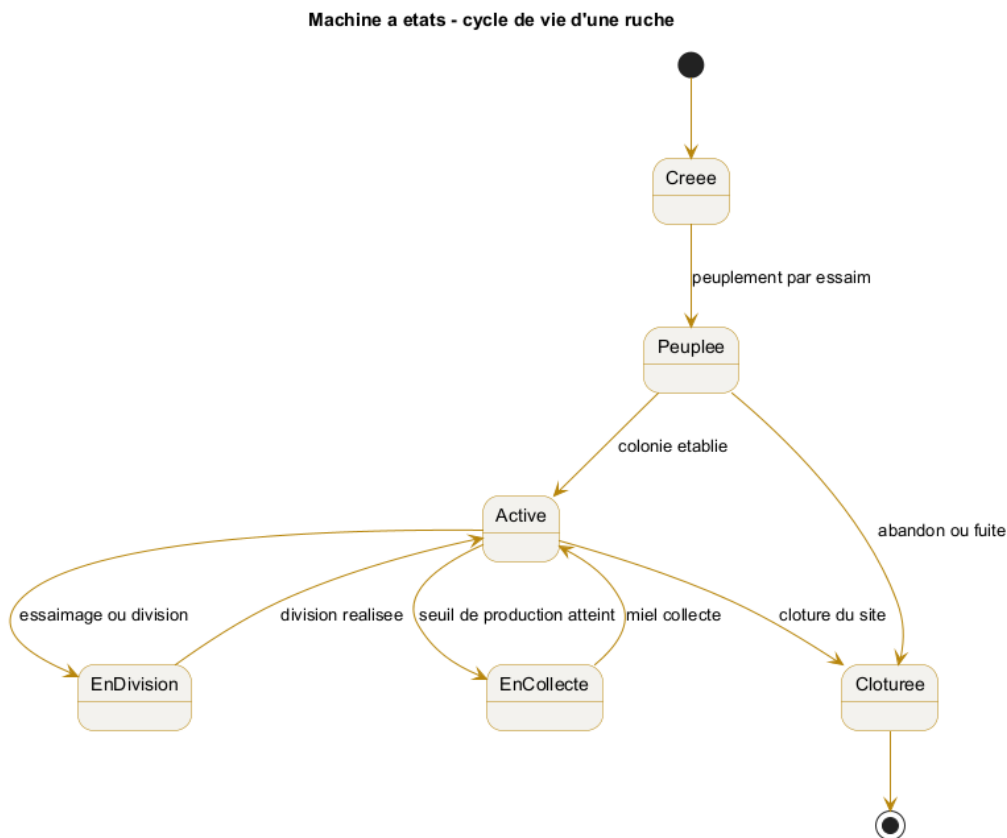


Figure 7.7. Machine à états du cycle de vie d’une ruche.

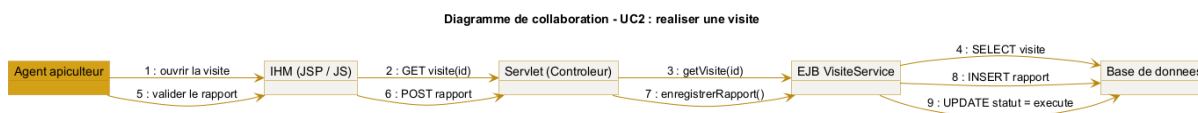


Figure 7.8. Diagramme de collaboration du cas d’utilisation UC2.

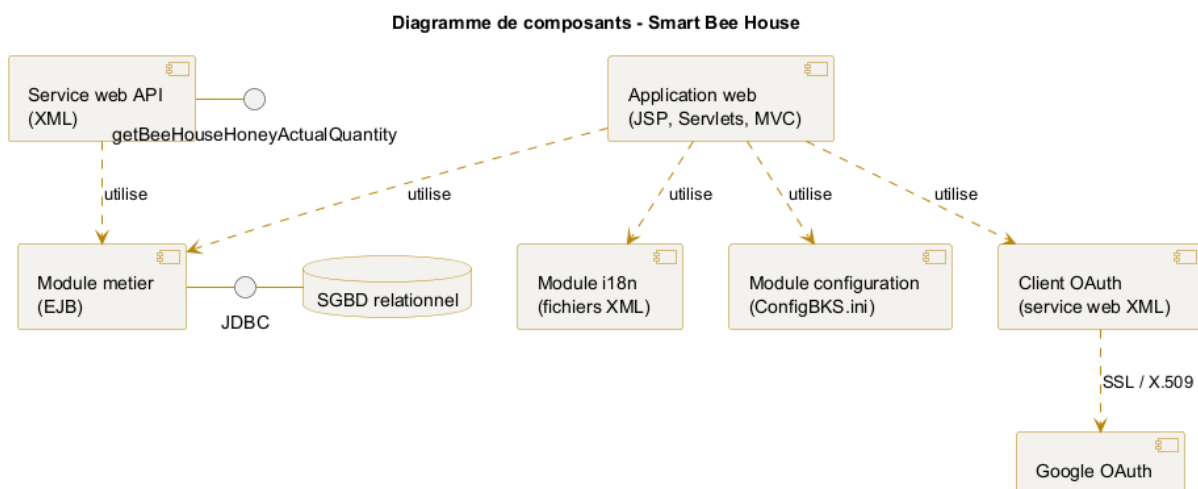


Figure 7.9. Diagramme de composants du système Zümm.

CHAPITRE 8

Contraintes et exigences techniques

8.1 Architecture

Architecture **scalable, évolutive, flexible, multi-niveaux et faiblement couplée**, s'appuyant sur **J2EE** comme infrastructure principale.

8.2 Exigences techniques imposées

Exigence	Description
MVC	Séparation modèle / vue / contrôleur.
Servlet	Traitement des requêtes côté serveur.
JSP	Front-end de présentation.
EJB	Composants métier.
JDBC	Accès aux données.
Internationalisation	Textes de l'IHM externalisés dans un fichier XML, ouverture à la traduction hors-ligne sans limite de langues, FR, EN et AR au minimum en démonstration.
Configuration	Paramétrage par fichier texte <code>ConfigBKS.ini</code> .
Sécurité inter-composants	Certificats X.509 et protocole SSL .
Authentification utilisateur	OAuth Google via un service web XML et son API.
API tierce	Service web XML exposant <code>float getBeeHouseHoneyActualQuantity(int beeHouseID)</code> pour récupérer la quantité de miel d'une ruche.
Front-end	JavaScript et frameworks libres, dans le respect de l'ergonomie imposée.

8.3 Exigences non fonctionnelles

- **Ergonomie, convivialité et simplicité** d'utilisation, consistance de l'interface et qualité du design infographique (importance capitale).
- **Évolutivité** : ouverture à l'ajout de langues, d'unités et d'indicateurs.
- **Interopérabilité** : services web interrogeables par des applications tierces.
- **Sécurité** : chiffrement des échanges et authentification forte.
- **Qualité de conception** : respect des principes **SOLID** et recours aux **design patterns** (patrons de conception) pour garantir un couplage faible et l'évolutivité, comme détaillé en annexe E.

Principes de conception retenus : l'architecture applique les cinq principes **SOLID** (responsabilité unique, ouvert/fermé, substitution de Liskov, ségrégation des interfaces, inversion de dépendance) et un ensemble de **design patterns** (Composite, State, Strategy, Observer, Command, Factory Method, Builder, Adapter, Facade, Singleton, Proxy). Leur application au système, justifiée « avant / après », est détaillée dans l'annexe E.

8.4 Questions de conception à couvrir

La conception devra répondre explicitement aux questions suivantes :

1. Quels sont les **inputs** (liste, dictionnaire de données et types) et **outputs** (liste, dictionnaire, types, formules et requêtes de calcul) du système ?
2. De quoi se compose la solution ?
3. Quels sont les utilisateurs types et leurs droits d'accès aux fonctionnalités ?
4. Comment le système réalise-t-il les opérations utilisateur ?
5. Comment le système est-il packagé et déployé ?
6. Comment les éléments interagissent-ils et dans quel ordre / chronologie ?
7. Pourquoi, comment et où telle technologie a-t-elle été utilisée ?

8.5 Justification des choix technologiques

Le tableau suivant répond au pourquoi, au comment et au où de chaque technologie.

Technologie	Pourquoi	Où
MVC	Séparer les responsabilités et faciliter la maintenance	Organisation générale de l'application
Servlet	Traiter les requêtes et orchestrer les traitements	Couche de contrôle
JSP	Produire les pages dynamiques	Couche de présentation
EJB	Encapsuler la logique métier et la rendre réutilisable	Couche métier
JDBC	Accéder à la base relationnelle de façon standard	Couche d'accès aux données

Technologie	Pourquoi	Où
XML i18n	Externaliser les textes pour la traduction hors-ligne	Module d'internationalisation
ConfigBKS.ini	Paramétrer le système sans recompilation	Module de configuration
OAuth Google	Authentifier les utilisateurs sans gérer les mots de passe	Accès au système
Service web API XML	Exposer les données aux applications tierces	Interface d'interopérabilité
X.509 et SSL	Chiffrer et authentifier les échanges	Communications inter-composants
JavaScript	Enrichir l'ergonomie du front-end	Couche de présentation

8.6 Packaging et déploiement

- **Packaging** : l'application est empaquetée sous forme d'archive Java d'entreprise (WAR pour la partie web, EAR si les EJB sont regroupés), incluant les fichiers XML d'internationalisation et le fichier `ConfigBKS.ini`.
- **Serveur** : déploiement sur un serveur d'applications J2EE fournissant conteneur de servlets et conteneur d'EJB.
- **Base de données** : SGBD relationnel accédé via JDBC.
- **Sécurité** : installation des certificats X.509 et activation de SSL pour les échanges, configuration de l'accès OAuth Google.
- **Configuration** : les seuils, unités et modules actifs sont ajustés dans `ConfigBKS.ini` sans redéploiement.

La répartition physique est illustrée par le diagramme de déploiement (figure 7.3).

CHAPITRE 9

Enveloppe budgétaire et ressources

9.1 Cadre

Le projet s'inscrit dans un cadre **académique (mini-projet)**. L'enveloppe budgétaire se traduit donc principalement en **ressources humaines, logicielles et matérielles** mobilisées, sans coût financier direct.

9.2 Ressources mobilisées

Ressource	Détail
Humaines	Équipe projet étudiante (conception, développement, tests, documentation).
Logicielles	Serveur d'applications J2EE, SGBD relationnel, IDE, outils UML, chaîne de compilation.
Matérielles	Postes de développement, capteurs simulés pour le volet indicateurs.
Externes	Compte OAuth Google, certificats X.509 (auto-signés en démonstration).

Contrainte académique majeure : tout usage de générateurs de code (ChatGPT, DeepSeek et dérivés) est proscrit et sera considéré comme une fraude à l'épreuve d'examen.

CHAPITRE 10

Délais et livrables

10.1 Livrables attendus

- Application fonctionnelle (code source J2EE + front-end).
- Plan de tests et résultats d'exécution.
- Dossier de conception (diagrammes UML).
- Rapport de projet, poster et présentation orale.

10.2 Grille d'évaluation (barème)

Critère	Points
Ergonomie	4
Fonctionnalités testées (plan de tests + exécution)	4
Technologies	4
MVC	0,5
Servlet	0,5
JSP front-end	0,25
Fichier XML et internationalisation	0,5
Fichier texte de configuration	0,25
Web service client OAuth	0,5
Web service API XML	0,5
JDBC	0,5
X.509 / SSL	0,25
EJB	0,25
Conception (UC, classes, objets, séquence, activité, état, interaction/collaboration, package, composant, déploiement)	4
Documentation	4
Rapport (structure, consistance, format)	2

Critère	Points
Poster	1
Présentation	1
Total	20

10.3 Diagrammes UML à produire

Diagrammes de : cas d'utilisation, classes, objets, séquence, activité, machine à états, interaction / collaboration, package, composant et déploiement. Le contenu attendu de chacun est détaillé au chapitre 7 portant sur la conception.

10.4 Planning prévisionnel

Le planning est exprimé en semaines relatives, avec les livrables intermédiaires associés à chaque phase.

Phase	Semaines	Livrable
Analyse et cahier des charges	S1 à S2	Cahier des charges validé
Conception (UML)	S3 à S4	Dossier de conception
Développement du noyau CRUD	S5 à S7	Modèle et gestion des entités
Tableaux de bord et alertes	S8 à S9	Calendrier, alertes et synthèses
Services web, sécurité, i18n	S10 à S11	OAuth, API, X.509 et SSL, XML
Tests et validation	S12	Plan de tests exécuté
Documentation et présentation	S13	Rapport, poster et soutenance

CHAPITRE 11

Plan de tests et validation

La stratégie de test couvre les fonctions métier, les contraintes de gestion et les exigences techniques. Chaque cas de test précise la fonction visée, les préconditions, les étapes et le résultat attendu.

11.1 Cas de tests

ID	Fonction	Étapes	Résultat attendu
TC01	CRUD ruche	Créer, lire, modifier, supprimer une ruche	Opérations conformes et persistées
TC02	Contrainte cadres	Ajouter un 11e cadre à un corps	Refus, maximum de 10 cadres
TC03	Contrainte hausses	Ajouter une 6e hausse	Refus, maximum de 5 hausses
TC04	Contrainte socle	Retirer le dernier cadre du corps	Refus, au moins 1 cadre
TC05	Planning de visite	Proposer puis approuver une visite	Statut approuvé enregistré
TC06	Rapport de visite	Remplir tous les champs et enregistrer	Rapport persisté, case exécutée
TC07	Alerte production	Dépasser le seuil de production	Alerte de collecte affichée
TC08	Alerte baisse	Simuler une chute d'effectif	Alerte d'inspection affichée
TC09	Calcul ROI	Saisir coûts et production	ROI calculé correctement
TC10	API miel	Appeler getBeeHouseHoneyActualQuantity	Quantité correcte retournée
TC11	Internationalisation	Basculer FR, EN puis AR	Textes traduits via le fichier XML

ID	Fonction	Étapes	Résultat attendu
TC12	Configuration	Modifier un seuil dans ConfigBKS.ini	Nouveau seuil pris en compte
TC13	Authentification	Se connecter par OAuth Google	Accès autorisé
TC14	Sécurité	Vérifier SSL et certificat X.509	Échanges chiffrés et validés
TC15	Export	Exporter les enregistrements	Fichiers CSV et TXT générés
TC16	Mode hors-ligne	Saisir sans réseau puis re-connecter	Synchronisation réussie

11.2 Matrice de traçabilité

Exigence	Cas de test
Gestion des entités et contraintes	TC01, TC02, TC03, TC04
Planification et rapports de visite	TC05, TC06
Tableaux de bord et alertes	TC07, TC08, TC09
Service web API	TC10
Internationalisation et configuration	TC11, TC12
Authentification et sécurité	TC13, TC14
Portabilité et mode hors-ligne	TC15, TC16

CHAPITRE 12

Référentiel métier et bonnes pratiques apicoles

Ce chapitre précise les connaissances métier qui fondent les règles de gestion, les seuils paramétrables et les indicateurs du système. Il constitue la base fonctionnelle sur laquelle reposent les tableaux de bord et les alertes.

12.1 Facteurs de production du miel

La production d'une colonie dépend directement de la météo, de la disponibilité du pollen et du flux de nectar. Lorsque le flux de nectar est élevé, la reine pond davantage, la population culmine et le rendement atteint son maximum. En période de pénurie alimentaire, la ponte et la population diminuent. Le système doit donc corrélérer la production mesurée aux périodes et aux saisons, ce qui justifie le filtrage par mois, semaine, saison et année des tableaux de bord.

12.2 Emplacement et configuration d'un site

Le choix de l'emplacement d'un site conditionne la productivité. Les valeurs de référence suivantes peuvent alimenter les champs et les contrôles de saisie du module de gestion des sites.

Critère	Valeur de référence
Proximité des ressources	Sources de nectar et de pollen à environ 1 km.
Accès à l'eau	Eau potable accessible, plusieurs litres consommés par jour et par colonie.
Éloignement des habitations	100 m en zone forestière, 200 m en zone arbustive, 300 m en zone ouverte.
Éloignement des cultures traitées	Au moins 3 à 4 km des cultures conventionnelles utilisant des pesticides.
Surélévation et inclinaison	Ruches surélevées à environ 1,5 m du sol et légèrement inclinées pour évacuer l'humidité.

Critère	Valeur de référence
Protection	Abri contre le soleil intense, le vent et les inondations.



Figure 12.1. Exemple de ruche installée sur un site : surélévation sur pieds et planche d'envol.

12.3 Types de ruches et rendement de référence

Type de ruche	Caractéristiques et rendement
Ruche traditionnelle à rayons fixes	6 à 10 kg de miel en rayons par saison, non recommandée en apiculture biologique.
Ruche à barrettes supérieures	Largeur standard de 32 à 35 cm, inspection indépendante des rayons.
Ruche à cadres (Langstroth, Est-Africaine)	Rayons à miel réutilisables plusieurs saisons après récolte.

Ces ordres de grandeur servent de base au paramétrage des seuils de production et à l'estimation du retour sur investissement par ruche.

12.4 Classification des modèles de ruches

On distingue deux grandes familles de ruches, dont la modélisation doit rester ouverte pour couvrir les usages locaux et internationaux.

Famille	Modèles représentatifs
Ruches à cadres (verticales)	Dadant, Langstroth, Voirnot, Layens, Aumônière, Nationale (GB), WBC, Tonelli.
Ruches traditionnelles sans cadres	Warré, Kényane TBH, Tanzanienne TBH, tronc, paille.
Autres modèles régionaux	Lucitana, Oksman, Smith, CDB, Duvauchelle, Jarry, Congres, Zanders, modèles slovènes.

En France, les modèles les plus courants sont les ruches Dadant, Langstroth et Voirnot. Toutes les ruches à cadres partagent une composition commune, avec des dimensions précises au millimètre près, ce qui conforte le modèle de données retenu.

Ancrage du modèle : le modèle de référence du projet est la ruche Dadant, dont le corps accueille jusqu'à 10 cadres. Cette valeur fonde la capacité maximale de 10 cadres par corps ou par hausse retenue dans les règles de gestion. Le système reste toutefois paramétrable pour supporter d'autres modèles.

12.4.1 Composition commune d'une ruche à cadres

Élément	Rôle
Plancher / planche d'envol	Base de la ruche et zone d'atterrissage des abeilles.
Corps (socle)	Compartiment principal du couvain, contient les cadres de base.
Hausse	Étage d'extension destiné à la récolte du miel.
Cadre	Support amovible des rayons de cire, installé sur un slot identifié.
Grille à reine	Sépare le corps des hausses et cantonne la reine dans le corps.
Couvre-cadre et toit	Ferment et protègent la ruche des intempéries.
Nourrisseur	Permet le ravitaillement en nourriture de la colonie.

12.5 Protocole d'inspection des colonies

Les inspections sont réalisées de préférence les jours ensoleillés et portent sur des points de contrôle précis qui structurent le rapport de visite.

- développement du couvain (oeufs, larves, cellules operculées)
- remplissage des alvéoles en miel et en pollen
- présence de parasites ou de maladies
- récolte de nectar, de pollen et de propolis

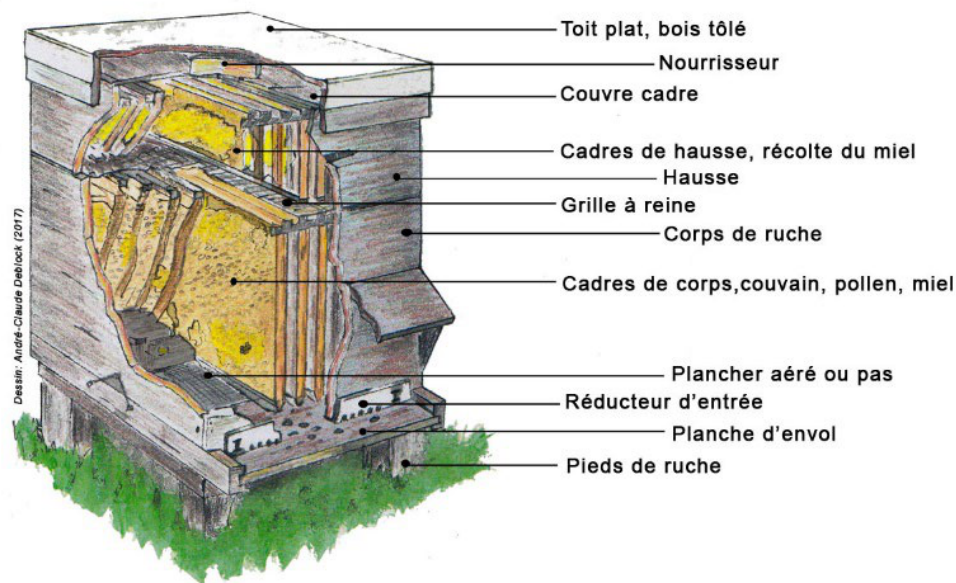


Figure 12.2. Coupe d'une ruche à cadres et rôle de chaque élément (toit, nourrisseur, couvercadre, hausse, grille à reine, corps, plancher).

Contrainte opérationnelle : une visite ne doit pas dépasser 45 minutes. Au-delà, l'agitation de la colonie augmente et se propage aux ruches voisines. Le champ *durée* du rapport de visite peut être contrôlé par rapport à cette limite de référence.

12.6 Essaimage et division des colonies

Un essaimage imminent se signale par la présence de cellules royales sur les bords des rayons. Quelques jours avant l'émergence d'une nouvelle reine, la vieille reine quitte la colonie avec la moitié des ouvrières. Le système doit donc permettre d'anticiper la division. Trois méthodes de référence sont prévues comme motifs d'intervention.

Méthode	Principe
Formation de nucléus	Retirer 1 à 2 rayons de couvain avec abeilles vers une ruche neuve.
Nucléus de reine	Transférer la reine avec un rayon sans cellules royales.
Essaimage artificiel	Déplacer la ruche mère et placer une nouvelle ruche à l'ancien emplacement avec 2 à 3 rayons de jeune couvain.

12.7 Indicateurs de bien-être et causes de baisse

Une colonie saine présente un nid à couvain bien développé aux différents stades, des alvéoles remplies de ressources, une activité de butinage visible et l'absence de parasites ou de maladies.

Ces critères alimentent l'évaluation qualitative de l'état de santé et le niveau de productivité noté de 1 à 3 dans le rapport de visite.

La fuite ou l'abandon d'une ruche résulte de perturbations excessives ou de conditions inadéquates (manque de nourriture ou d'eau, températures extrêmes). Laisser une réserve de miel à la colonie lors de la récolte réduit ce risque. Ces phénomènes justifient les alertes de chute soudaine d'effectif ou de production et les seuils paramétrables associés.

12.8 Incidence sur les seuils paramétrables

Paramètre système	Ancrage métier
Seuil de collecte de miel	Rendement de référence par type de ruche et par saison.
Seuil d'alerte de baisse	Chute anormale liée à une fuite, une maladie ou une pénurie alimentaire.
Durée maximale de visite	Limite de 45 minutes par inspection.
Distances de sécurité du site	100, 200 ou 300 m selon la zone, 3 à 4 km des cultures traitées.
Périodes d'analyse	Corrélation production et saison (flux de nectar).

ANNEXE A

Annexe : Structure physique d'une ruche

Pour mémoire, une ruche à cadres amovibles se compose, de bas en haut, des éléments suivants : plancher / planche d'envol, corps (socle) contenant les cadres de couvain, grille à reine, une ou plusieurs hausses contenant les cadres de récolte du miel, couvre-cadre et toit. Cette structure justifie les règles de composition (corps obligatoire, jusqu'à 5 hausses, 1 à 10 cadres par élément) retenues dans le modèle métier.

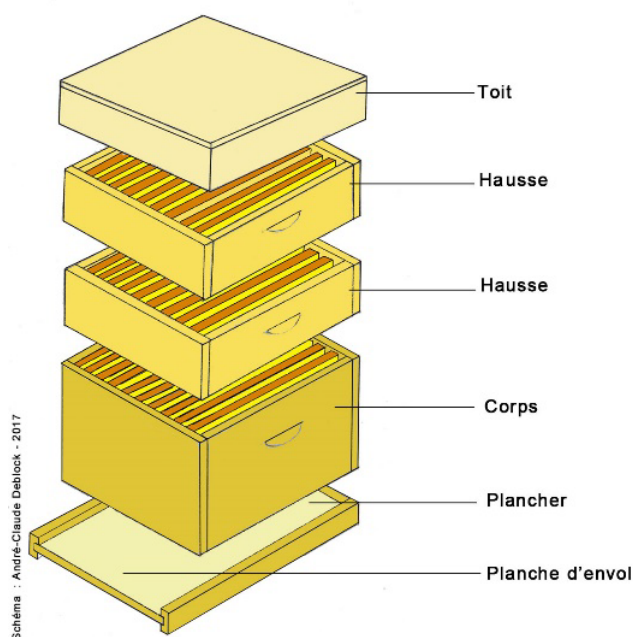


Figure A.1. Vue éclatée d'une ruche à cadres : toit, hausses, corps, plancher et planche d'envol.

ANNEXE B

Glossaire

Terme	Définition
Socle ou corps	Compartiment de base de la ruche contenant les cadres de couvain.
Hausse	Étage d'extension ajouté au corps pour la récolte du miel.
Cadre	Support amovible des rayons de cire, installé sur un slot.
Slot	Emplacement identifié d'un cadre dans un corps ou une hausse.
Essaim	Colonie d'abeilles peuplant une ruche.
Essaimage	Départ d'une partie de la colonie avec la reine pour fonder une nouvelle colonie.
Nucléus	Petite colonie constituée pour diviser un essaim ou élever une reine.
Reine	Unique femelle reproductrice de la colonie.
Couvain	Ensemble des oeufs, larves et nymphes de la colonie.
Butinage	Activité de collecte du nectar et du pollen par les abeilles.
Propolis	Substance résineuse récoltée par les abeilles.
Grille à reine	Grille qui cantonne la reine dans le corps et sépare les hausses.
Nourrisseur	Dispositif de ravitaillement en nourriture de la colonie.
Seuil paramétrable	Valeur de référence configurable déclenchant une alerte.
ROI	Retour sur investissement, rapport entre production valorisée et coûts.
CRUD	Opérations de création, lecture, mise à jour et suppression.
i18n	Internationalisation, adaptation aux langues et aux systèmes d'unités.
EJB	Composant métier de l'infrastructure J2EE.
Servlet	Composant serveur traitant les requêtes web.
JSP	Page serveur produisant l'affichage dynamique.
JDBC	Interface d'accès aux bases de données relationnelles.

Terme	Définition
OAuth	Protocole d'autorisation permettant l'authentification déléguée.
X.509 et SSL	Certificats et protocole assurant des échanges chiffrés et authentifiés.

ANNEXE C

Références et sources

Le présent cahier des charges s'appuie sur les sources suivantes.

Méthode d'élaboration

- Manager-GO, *Élaborer un cahier des charges*.
<https://www.manager-go.com/gestion-de-projet/dossiers-methodes/elaborer-un-cdc>

Principes de conception logicielle

- Alex so yes, *Les principes SOLID*.
<https://alexsoyes.com/solid/>
- Refactoring Guru, *Catalogue des design patterns*.
<https://refactoring.guru>

Référentiel métier apicole

- Organic Africa, *Amélioration de la production de miel*.
<https://www.organic-africa.net/fr/agriculture-biologique/agriculture-biologique/animaux/apiculture/amelioration-de-la-production-de-miel.html>
- Au Bon Miel, *Les ruches*.
<https://www.aubonmiel.com/les-ruches/>

Fonctionnalités des solutions du marché

- HiveTracks, application de gestion apicole.
<https://www.hivetracks.com/>
- Apiary Book, solution de gestion de rucher.
<https://www.apiarybook.com/>
- BeePlus, gestionnaire d'apiculture (App Store).

Surveillance connectée (precision beekeeping)

- Arnia, surveillance acoustique et pesée des ruches.
<https://www.arnia.co/>

- BroodMinder, capteurs de température, d'humidité et de poids.

<https://broodminder.com/>

- BeeHero, plateforme IoT et analyse par intelligence artificielle.

<https://www.beehero.io/>

Retours d'expérience et limites des solutions

- Beekeeping Diary, *Best Beekeeping Apps 2026 (comparatif)*.

<https://beekeeping-diary.eu/blog/en/best-beekeeping-apps-2026/>

- HiveBook, *Free HiveTracks Alternatives*.

<https://hivebook.app/blog/hivetracks-alternatives-free>

- Beesource Forums, *Problems with Hive Tracks*.

<https://www.beesource.com/threads/problems-with-hive-tracks.249656/>

- ApiTech World, *Smart Hive Monitoring, a Beekeeper's Reality Check (2026)*.

<https://apitechworld.com/smart-hive-monitoring-a-beekeepers-reality-check-2026-review/>

ANNEXE D

Annexe - Choix technologique : academique et vision marche

Cette annexe repond a la question du *pourquoi* et du *comment* du langage de developpement. Elle confronte le socle **J2EE** impose par le present cahier des charges a l'ecosysteme **Spring Boot**, devenu le standard du developpement Java d'entreprise, et documente une trajectoire de modernisation sans changement de langage.

D.1 Positionnement

Le systeme **Zümm** est implemente en **J2EE (Java EE)** conformement aux exigences techniques (chapitre 7 et grille d'évaluation) : MVC, Servlets, JSP, EJB, JDBC, internationalisation externalisee, services web securises (OAuth, X.509 / SSL) et API interrogeable par des applications tierces. Ce socle materialise les competences fondamentales du parcours DSI : maitrise des couches d'un systeme d'information, separation des responsabilites et interoperabilite.

Constat marche : les briques Java EE historiques (JSP, Servlets « nus » et EJB) sont aujourd'hui considerees comme *heritees (legacy)*. Les recrutements sur les nouveaux systemes d'information privilegient tres majoritairement l'ecosysteme **Spring Boot**. La presente annexe conserve le langage **Java** et demontre, techno par techno, l'equivalent moderne de chaque exigence.

D.2 Tableau de correspondance : coeur technique

Le tableau suivant met en regard chaque technologie imposee (et notee) avec son equivalent Spring Boot, ainsi que la nature concrete du changement.

J2EE (impose)	Spring Boot	Ce qui change concretement
MVC (manuel)	Spring (@Controller, @RequestMapping)	MVC Le pattern reste ; le routage devient declaratif par annotations au lieu du <code>web.xml</code> .

J2EE (impose)	Spring Boot	Ce qui change concrètement
Servlet (<code>HttpServletRequest</code> , <code>doGet/doPost</code>)	<code>@RestController</code> / <code>@Controller</code>	Plus de <code>extends HttpServlet</code> : une methode annotee = un endpoint. Le <code>DispatcherServlet</code> orchestre en arriere- plan.
JSP (presenta- tion)	Thymeleaf (server- side) ou React / An- gular (SPA)	Templating moderne sans scriptlets Java ; pour un effet marche fort, API REST + front decouple.
EJB (<code>@Stateless</code> , conteneur EJB)	Spring <code>@Service</code> / <code>@Component</code>	Meme role (logique metier, injection, transactions) sans conteneur EJB lourd ; <code>@Transactional</code> gere les transactions.
JDBC (SQL manuel, <code>ResultSet</code>)	Spring Data JPA / Hibernate	Le mapping objet-relationnel remplace le SQL a la main ; <code>JpaRepository</code> genere les CRUD (<code>JdbcTemplate</code> reste disponible).
18n par fichier XML	<code>messages_fr/en/ar.properties</code> + <code>MessageSource</code>	Principe d'externalisation des textes, format <code>.properties</code> standard, toujours FR / EN / AR.
ConfigBKS.ini	<code>application.yml</code> + <code>@ConfigurationProperties</code>	Parametrage sans recompilation, injectable par profil ; les seuils et unites deviennent des propriétés sans types.
OAuth Google (WS XML mai- son)	Spring Security OAuth2 Client	Login Google en quelques lignes de configu- ration au lieu d'un client OAuth ecrit a la main, et plus sur.
Web service API XML <code>getBeeHouseHoneyAcquaPlantSyngger</code>	API REST JSON (<code>@GetMapping</code>) + Swagger	JSON au lieu de XML, documentation auto-generee ; XML reste possible via <code>produces = APPLICATION_XML</code> .
X.509 / SSL (manuel)	Spring Boot SSL (<code>server.ssl.*</code>) + Keystore	Activation HTTPS declarative, memes cer- tificats X.509, configuration triviale.
JavaScript (front libre)	React / Vue / Angu- lar (ou Thymeleaf + JS)	Front decouple consommant l'API REST, parite web et mobile simplifiee.

D.3 Tableau de correspondance : infrastructure et deployment

J2EE	Spring Boot	Benefice
Serveur d'applications lourd (WildFly, GlassFish, WebLogic)	Tomcat / Jetty embarque	L'application se lance seule : <code>java -jar app.jar</code> , plus de serveur a installer.
Packaging WAR / EAR	JAR executable (fat-jar)	Un seul artefact autonome, ideal pour le cloud et Docker.
Deploiement manuel sur conteneur	Docker + <code>docker-compose</code>	Alignement DevOps du marche, integration continue facilitee.
SGBD relationnel via JDBC	PostgreSQL / MySQL via JPA + Flyway	Schema versionne, reproductible entre environnements.
Gestion des dependances manuelle (Ant)	Maven / Gradle + Spring Boot Starters	Une dependance = un starter (<code>-web</code> , <code>-security</code> , <code>-data-jpa</code>).

D.4 Correspondance des couches

La vue en couches faiblement couplees exiguee au chapitre 7 (figure 7.2) est **integralement preservee** : aucune couche ne disparaît, seule la mise en oeuvre est modernisee.

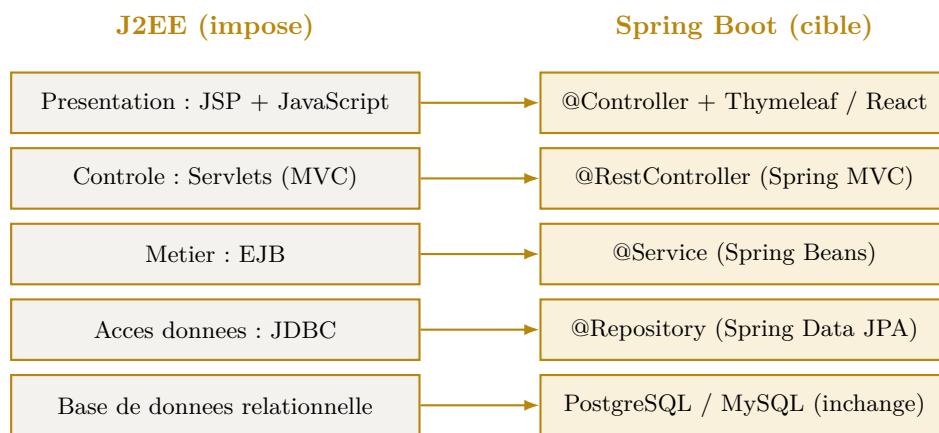


Figure D.1. Correspondance des couches J2EE vers Spring Boot : architecture preservee, mise en oeuvre modernisee.

Les modules transverses suivent la meme logique : `i18n XML` → `MessageSource`, `ConfigBKS.ini` → `application.yml`, service web XML → API REST + OpenAPI.

D.5 Argumentaire : conformite academique et vision marche

Zümm repose, conformement au cahier des charges, sur une architecture **J2EE multi-niveaux, evolutive et faiblement couplee** : MVC, Servlets, JSP, EJB, JDBC, internationalisation par fichier externalise, services web securises (OAuth, X.509 / SSL) et API interrogeable par des applications tierces. Ce socle a ete retenu et implemente **tel qu'exige**, car il constitue le referentiel

d'évaluation du projet et materialise les competences fondamentales du parcours DSI : maitrise des couches d'un systeme d'information, separation des responsabilites et interoperabilite.

Une analyse **orientee marche** conduit toutefois a un constat important. Les briques Java EE historiques (JSP, Servlets « nus » et EJB) sont aujourd'hui considerees comme *heritees*. Les recrutements sur les nouveaux systemes d'information privilegient tres majoritairement l'ecosysteme **Spring Boot**, devenu le standard *de facto* du developpement Java d'entreprise. Ignorer cette realite reviendrait a livrer une solution techniquement conforme mais **decalee des pratiques professionnelles actuelles**.

Nous avons donc adopte une **demarche a deux niveaux de lecture**, sans jamais sortir du langage **Java**, ce qui garantit la continuite des competences acquises :

1. **Niveau conformite** : le livrable respecte integralement les technologies imposees et la grille d'évaluation. Les diagrammes de conception (sequence Agent → JSP → Servlet → EJB → JDBC, couches, deploiement) refletent fidelement cette architecture.
2. **Niveau modernisation** : pour chaque technologie imposee, son equivalent Spring Boot est demontre (Servlet → @RestController, EJB → @Service, JDBC → Spring Data JPA, XML i18n → MessageSource, ConfigBKS.ini → application.yml, WS XML → API REST + OpenAPI, X.509 / SSL → configuration SSL declarative). Cette correspondance prouve que **l'architecture cible reste identique** : seule la mise en oeuvre gagne en concision, en securite et en maintenabilite.

Ce choix presente un troisieme avantage, en phase directe avec les debouches du parcours DSI. Le cahier des charges prevoit un **volet capteurs et analyse predictive** (detection d'essaimage, traitement acoustique). Ce module se prete naturellement a un **microservice Python** (pandas, scikit-learn) expose au back-end Java via REST, illustrant concretement le metier d'**analyste de donnees** cite au plan d'etudes et l'ouverture d'une **architecture faiblement couplee** a l'heterogeneite technologique.

Conclusion : le projet satisfait pleinement les exigences academiques (J2EE) tout en documentant sa **trajectoire de modernisation** (Spring Boot et microservice data). Cette double lecture transforme une contrainte pedagogique en atout professionnel : elle demontre non seulement la maitrise des fondamentaux du systeme d'information, mais aussi la capacite a faire evoluer une architecture vers les standards recherches sur le marche de l'emploi en 2026.

ANNEXE E

Annexe - Principes SOLID et design patterns

Cette annexe répond à la question du *comment* de la qualité logicielle. Elle documente l'application des cinq principes **SOLID** et d'un ensemble de **design patterns** (patrons de conception) au système **Zümm**, en mettant chaque décision en regard de la conception initiale (*avant*) et de la conception refactorisée (*après*). L'objectif est une architecture **multi-niveaux, évolutive et faiblement couplée**, conforme aux exigences du chapitre 7.

Sources méthodologiques : les définitions retenues suivent la présentation des principes SOLID de *alexsoyes.com* et le catalogue de patrons de *refactoring.guru*, adaptés au domaine apicole du projet.

E.1 Les cinq principes SOLID

Principe	Avant (conception initiale)	Après (refactorisée)
S – Responsabilité unique	L'entité Ruche porte la méthode de calcul <code>getBeeHouseHoneyActualQuantity()</code> et le rapport devient <code>RapportVisite</code> . elle mélange l'état (données) et la logique métier; Visite regroupe données et champs de rapport.	Le calcul est extrait dans <code>ServiceMielImpl</code> (via le Composite) et le rapport devient <code>RapportVisite</code> . Chaque classe n'a plus qu'une seule raison de changer.
O – Ouvert / ferme	La détection d'alertes et la conversion d'unités reposeraient sur des <code>if / switch</code> sur <code>typeSeuil</code> et <code>unite</code> : ajouter une règle ou une unité impose de modifier le code existant.	<code>RegleAlerte</code> (Strategy) et <code>ConvertisseurUnite</code> : une nouvelle règle ou unité = une nouvelle classe, sans toucher au <code>GestionnaireAlertes</code> . Ouvert à l'extension, ferme à la modification.

Principe	Avant (conception initiale)	Après (refactoree)
L – Substitution de Liskov	Le diagramme de cas d'utilisation generalise l'agent superviseur a partir de l'agent apiculteur ; transpose tel quel en heritage de classes, un sous-type restrictif casserait la substituabilite.	Corps et Hausse heritent de Compartment en respectant integralement son contrat (capacite 1..10, <code>getPoidsMiel</code>). Le role de l'agent passe par l'enumeration Role + permissions (composition) plutot qu'un heritage restrictif.
I – Segregation des interfaces	Un service unique « fourre-tout » (CRUD + visites + tableaux de bord + API miel) obligerait l'application tierce a dependre de methodes inutilisees.	Interfaces cibles : ServiceRequeteMiel (l'API tierce ne voit que <code>getBeeHouseHoneyActualQuantity</code>), ServiceVisite , ServiceTableauDeBord . Chaque client ne depend que de son contrat.
D – Inversion de dependance	La couche metier (EJB) appelle directement JDBC (implementation concrete) : couplage fort a la base et aux ResultSet .	La couche metier depend d'abstractions RucheRepository / VisiteRepository (pattern DAO), implementees par RucheRepositoryJdbc . Tests par mock et migration JDBC → Spring Data JPA facilitees (cf. annexe D).

E.2 Design patterns retenus

Le tableau suivant recense les patrons appliques, leur categorie (creationnel, structurel, comportemental) et la couche concerne, avec la justification *avant / apres*.

Pattern	Avant	Après
Composite <i>structurel – metier</i>	Le calcul du miel parcourt manuellement les collections de cadres avec du code de sommation duplique.	<code>ElementRuche.getPoidsMiel()</code> est resolu recursivement (Ruche → Compartment → Cadre) : traitement uniforme de l'arborescence.
State <i>comportemental – metier</i>	Le statut de la ruche serait teste par des switch disperses (creee, active, en collecte, cloturee, ...).	EtatRuche et ses etats concrets encapsulent les transitions autorisees ; alignement direct sur le diagramme de machine a etats.
Strategy <i>comportemental – metier</i>	Les regles d'alerte, la conversion d'unites et le ROI seraient codes en dur.	Familles d'algorithmes interchangeables (RegleAlerte , ConvertisseurUnite), injectables et testables isolément.

Pattern	Avant	Après
Observer <i>comportemental</i> – <i>metier</i>	Le code de mesure appellerait directement les tableaux de bord (couplage fort).	GestionnaireAlertes notifie les Observateur abonnées (tableau de bord d'alertes, notification du responsable) au franchissement d'un seuil.
Command <i>comportemental</i> – <i>controle</i>	Les operations terrain (collecte, division, changement de reine) sont des appels directs, non rejouables hors-ligne.	Chaque operation devient une CommandeVisite ; la FileCommandesHorsLigne rejoue les commandes a la reconnexion (mode hors-ligne + synchronisation differee).
Factory Method <i>creationnel</i> – <i>metier</i>	Les ruches sont creees par des new disperses avec initialisation manuelle des compartiments.	FabriqueRuche.creer(modele) produit une ruche preconfiguree selon le modele (Dadant, Langstroth...) avec corps et capacites coherents.
Builder <i>creationnel</i> – <i>presentation</i>	Le rapport de visite, aux nombreux champs optionnels, imposerait un constructeur telescopique ou des setters epars.	ConstructeurRapport construit RapportVisite de facon fluide (constatations, actions, photos, evaluations).
Adapter <i>structurel</i> <i>donnees</i>	Les capteurs heterogenes (formats et unites differents) seraient traites au cas par cas.	AdaptateurCapteur uniformise chaque source en Mesure avec unites parametrables (volet automatise).
Facade <i>structurel</i> <i>controle</i>	Le Servlet orchestre directement tous les services metier.	FacadeApplication offre un point d'entree simplifie aux Servlets par-dessus les sous-systemes.
Singleton <i>creationnel</i> – <i>transverse</i>	Le fichier ConfigBKS.ini et les belles il8n seraient relus en plusieurs instances.	Configuration et GestionnaireIl8n garantissent une instance unique partagee.
Proxy <i>structurel</i> <i>services web</i>	L'acces a l'API exposerait directement le service metier.	Un ProxySecurite controle l'authentification OAuth et le chiffrement SSL / X.509 avant de deleguer au service.

E.3 Vue de conception refactorée

La figure E.1 synthetise les principaux patrons dans un diagramme de classes de conception : le Composite (composition de la ruche et calcul du miel), le State (cycle de vie), les services segreges (ISP), la persistance par abstraction (DIP / Repository), le couple Strategy + Observer (alertes) et le trio Factory / Builder / Command.

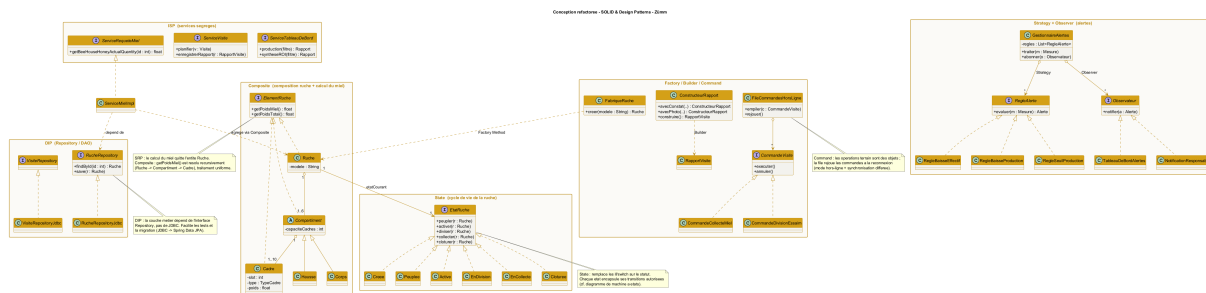


Figure E.1. Diagramme de classes refactoré : application de SOLID et des design patterns au système Zümm.

Correspondance avec les couches : les **Repository** appartiennent à la couche d'accès aux données (JDBC), les **Service** et le domaine (**Composite**, **State**, **Strategy**, **Observer**, **Command**) à la couche métier (EJB), la **Facade** et le **Proxy** à la couche de contrôle (Servlets), le **Builder** de rapport à la présentation (JSP). L'architecture en couches faiblement couplées du chapitre 7 est ainsi renforcée, sans être modifiée.

E.4 Illustration dynamique : patrons Observer et Command

Deux diagrammes de séquence détaillent le comportement des patrons les plus structurants du système.

E.4.1 Observer (+ Strategy) : déclenchement d'une alerte

La figure E.2 montre comment une mesure recue par le **GestionnaireAlertes** (sujet) est évaluée par une règle interchangeable (*Strategy*) puis, en cas de dépassement, propagée aux observateurs abonnés (tableau de bord d'alertes, notification du responsable). Ajouter une règle ou un observateur n'impose aucune modification du gestionnaire (principe ouvert/ferme).

E.4.2 Command : mode hors-ligne et synchronisation différée

La figure E.3 illustre la saisie d'une opération terrain sans réseau : l'opération est encapsulée dans une **CommandeVisite** empilée dans la **FileCommandesHorsLigne**. Au retour du réseau, la file rejoue (**executer**) chaque commande, ou la compense (**annuler**) en cas de conflit, réalisant la synchronisation différée exigée par le mode hors-ligne.

E.4.3 State : cycle de vie de la ruche

La figure E.4 montre la Ruche (contexte) qui délègue chaque opération à son état courant. L'état décide de la transition (activation, collecte, retour à l'état actif) ou refuse une opération interdite dans l'état en cours – ici un peuplement demande sur une ruche déjà active. Le comportement dépend ainsi de l'état sans aucun **switch** sur le statut, en cohérence avec le diagramme de machine à états (figure 7.7).

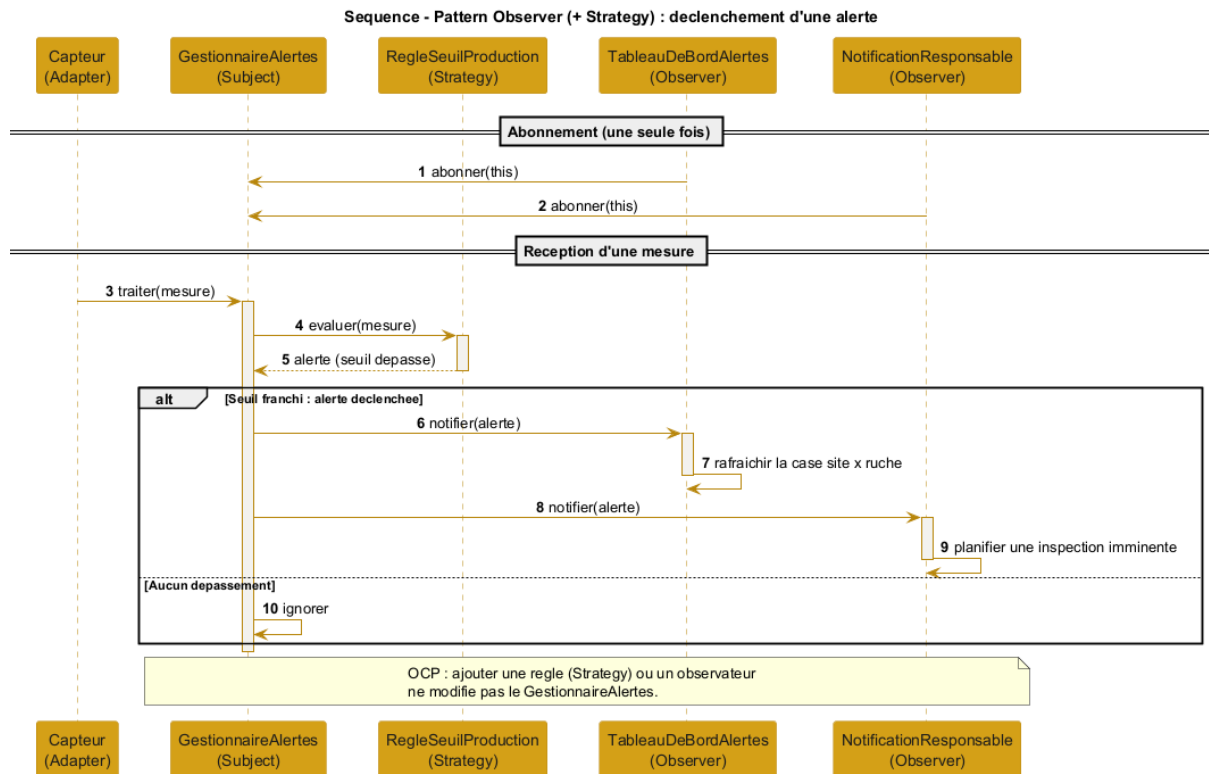


Figure E.2. Sequence du pattern Observer (+ Strategy) : flux de declenchement d'une alerte.

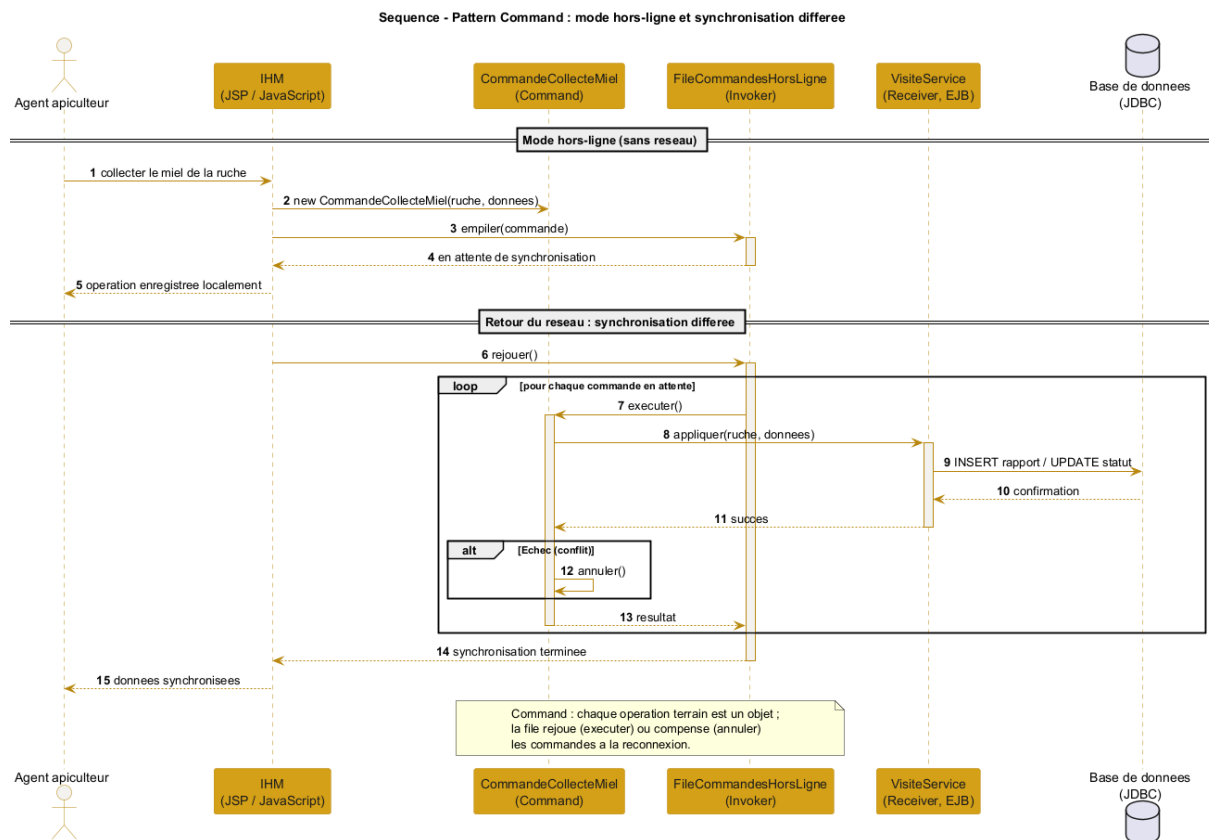


Figure E.3. Sequence du pattern Command : mode hors-ligne et synchronisation differee.

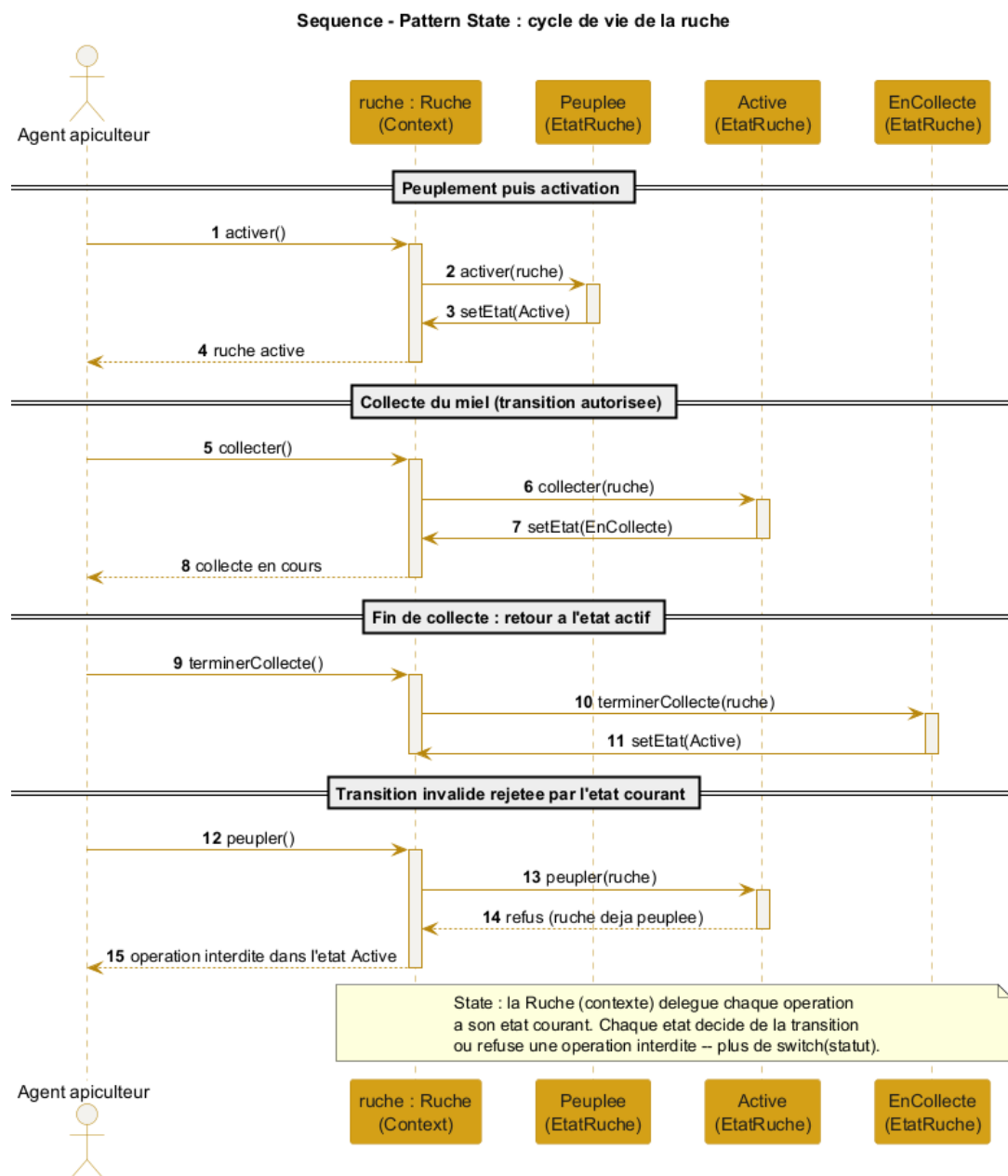


Figure E.4. Sequence du pattern State : delegation des transitions du cycle de vie de la ruche.

E.4.4 Composite : calcul de la quantité de miel

La figure E.5 détaille la résolution recursive de `getPoidsMiel()` : le client (`ServiceMielImpl`) interroge la ruche, qui propage l'appel a chaque compartiment, puis a chaque cadre (feuille). Seuls les cadres de type MIEL contribuent, et la tare est retranchée au niveau de la ruche. Le client ignore la structure interne; ce calcul realise la methode `getBeeHouseHoneyActualQuantity` exposee par le service web.

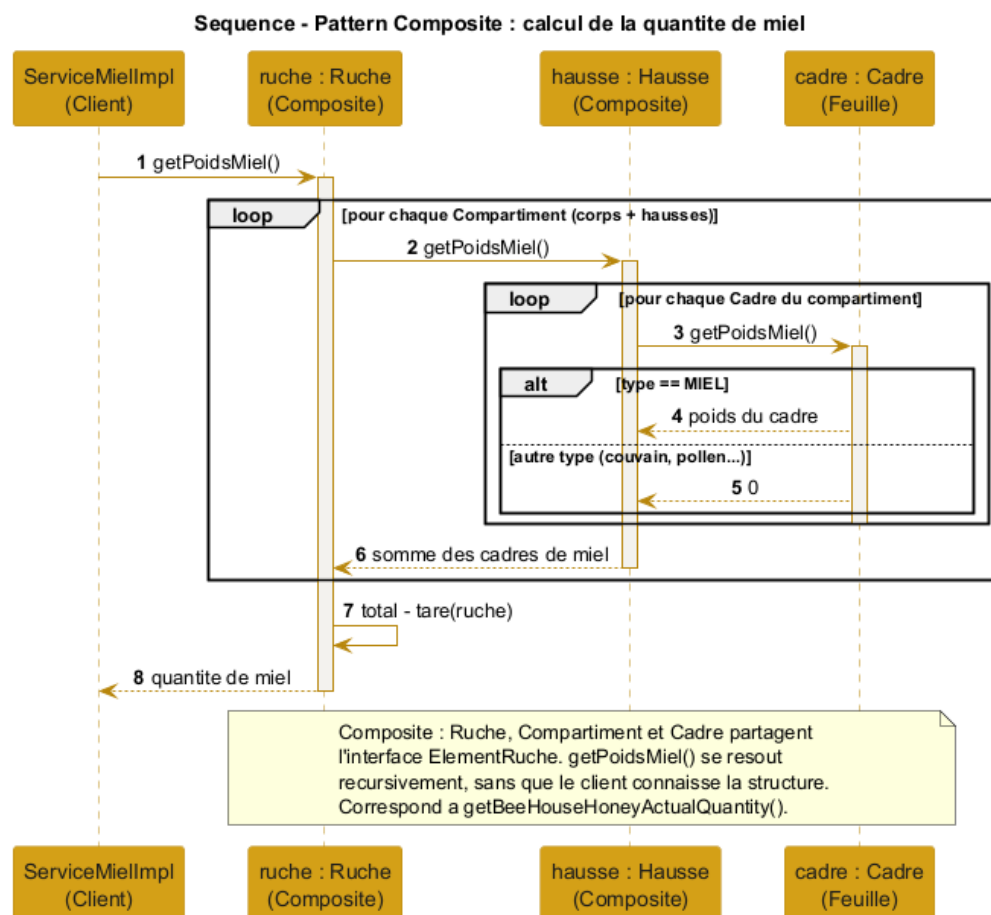


Figure E.5. Sequence du pattern Composite : calcul recursif de la quantité de miel d'une ruche.

E.4.5 Strategy : conversion d'unités heterogenes

La figure E.6 montre la normalisation d'une mesure exprimee dans une unite quelconque : le `RegistreConvertisseurs` (contexte) selectionne la strategie correspondant a l'unité (`ConvertisseurLbVersKg`) qui ramene la valeur a l'unité de reference avant comparaison au seuil. Une nouvelle unite se traduit par une nouvelle strategie, sans modifier le contexte.

E.4.6 Adapter : ingestion de capteurs heterogenes

La figure E.7 illustre l'integration d'un capteur du marche : l'`AdaptateurBroodMinder` traduit l'API proprietarie du capteur en une `Mesure` normalisee (unites converties, horodatage, geolocalisation), exposant l'interface uniforme `Capteur` attendue par la couche metier. Les capteurs heterogenes deviennent ainsi interchangeables.

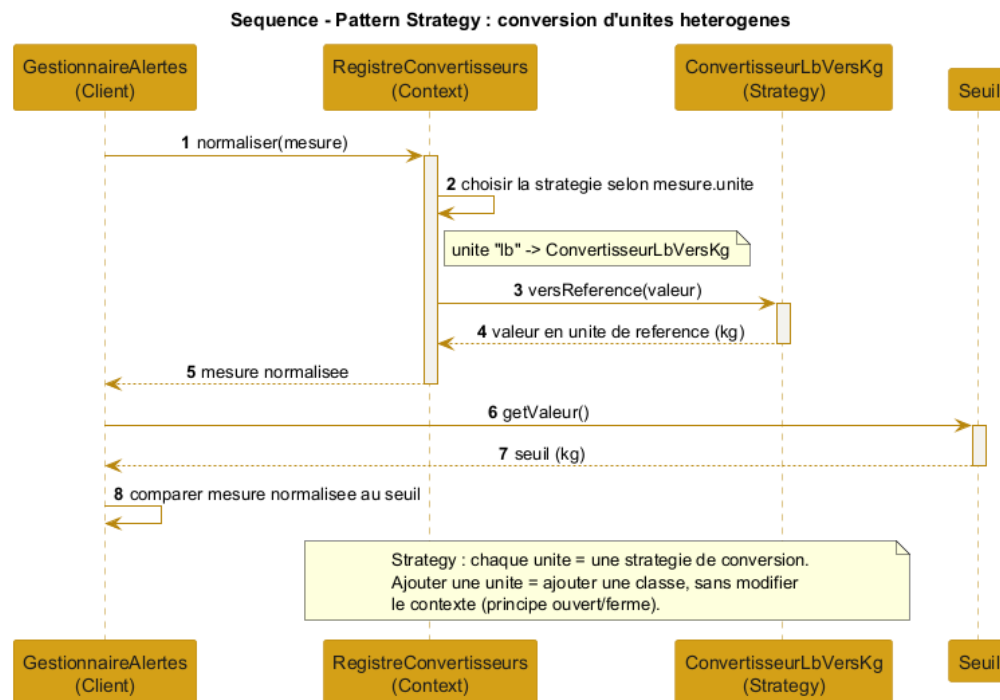


Figure E.6. Sequence du pattern Strategy : conversion d'unités hétérogènes vers l'unité de référence.

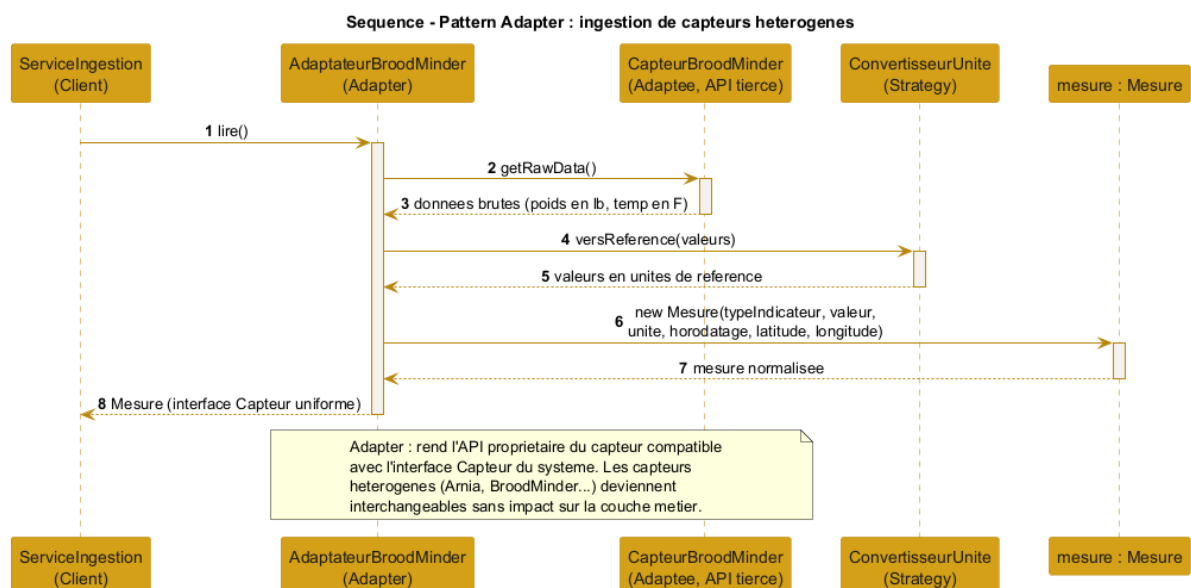


Figure E.7. Sequence du pattern Adapter : ingestion de capteurs hétérogènes (volet automatisé).

E.5 Repartition des patrons par couche

La figure E.8 synthetise la localisation de chaque patron dans l'architecture multi-niveaux : le *Builder* en presentation ; la *Facade*, le *Proxy* et la file de *Command* en controle ; le domaine (*Composite*, *State*, *Observer*, *Strategy*, *Factory Method*) et les services segregés (*ISP*) en metier ; le *Repository* (*DAO* / *DIP*) et l'*Adapter* en acces aux donnees ; enfin les *Singleton* de configuration et d'internationalisation ainsi que la *Strategy* de conversion d'unites en couche transverse. Cette vue confirme que les patrons renforcent l'architecture en couches faiblement couplees sans en alterer la structure.

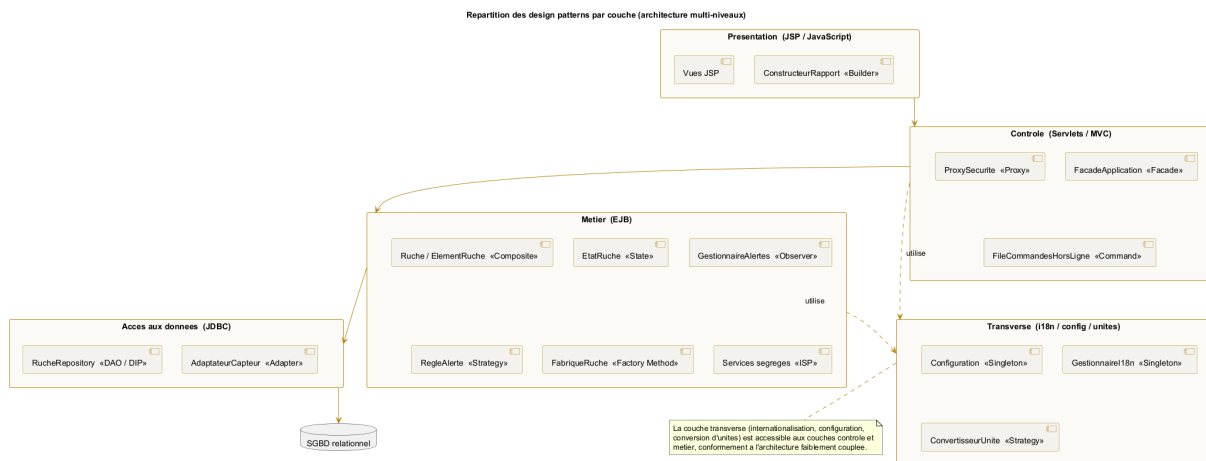


Figure E.8. Repartition des design patterns par couche de l'architecture multi-niveaux.